

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 14 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127
128 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
129 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
130 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
131
132 static constexpr int kWarpSize = 32;
133
134 // =====
135 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
136 // =====
137
138 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
139 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
140 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
141 // 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
142 // kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
143 // 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时, 只有同 segment 的 lane 参与通信
144 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
145 //
146 // 初始: 每个 lane 持有自己的值 v0..v7
147 // lane:  0    1    2    3    4    5    6    7
148 // val:  v0   v1   v2   v3   v4   v5   v6   v7
149 //
150 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
151 //
152 //      ┌──────────┐
153 // 对:   (0,4) (1,5) (2,6) (3,7)
154 //
155 // lane:  0    1    2    3    4    5    6    7
156 // val:  v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
157 //
158 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
159 //
160 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
161 // 对:   (0,2) (1,3) (4,6) (5,7)
162 //      ─── 前4个一组 ───      ─── 后4个一组 ───
163 //
164 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
165 // val:   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
166 //      = v0+v2+v4+v6 ... (逐步归约)
167 //
168 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
169 //
170 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
171 // 对:   (0,1) (2,3) (4,5) (6,7)
172 //
173 // lane:  0    1    2    3    4    5    6    7
174 // val:   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
175 //
176 // mask=0: 循环终止, 归约完成。
177 //
178 // 关键性质:
179 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
180 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
181 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
182 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
183 // - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width:
184 //   当 kWarpWidth=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
185 template <const int kWarpWidth = kWarpSize, typename T = float>
186 __device__ __forceinline__ T warp_reduce_sum(T val) {
187 #pragma unroll
188     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
189         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
190     }
191     return val;
192 }

```

```

189 }
190
191 // Warp Reduce Max — generic
192 template <const int kWarpWidth = kWarpSize, typename T = float>
193 __device__ __forceinline__ T warp_reduce_max(T val) {
194 #pragma unroll
195     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
196         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
197     }
198     return val;
199 }
200
201 // =====
202 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
203 // =====
204
205 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
206 // 两级归约流程:
207 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
208 // 2. warp leader (lane=0) 写入 shared memory
209 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
210 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
211 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
212 template <const int kNumThreads = 256>
213 __device__ float block_reduce_sum(float val) {
214     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
215     int warp = threadIdx.x / kWarpSize;
216     int lane = threadIdx.x % kWarpSize;
217     __shared__ float shared[kNumWarps];
218
219     float value = warp_reduce_sum<kWarpSize>(val);
220     if (lane == 0)
221         shared[warp] = value;
222     __syncthreads();
223     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
224     value = warp_reduce_sum<kNumWarps>(value);
225     // 关键: broadcast 结果到所有线程, 后续用 result
226     // 做除法等操作时所有线程都能拿到
227     value = __shfl_sync(0xffffffff, value, 0, 32);
228     return value;
229 }
230
231 // Block Reduce Max — FP32 (增强版, 带 broadcast)
232 template <const int kNumThreads = 256>
233 __device__ float block_reduce_max(float val) {
234     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
235     int warp = threadIdx.x / kWarpSize;
236     int lane = threadIdx.x % kWarpSize;
237     __shared__ float shared[kNumWarps];
238
239     float value = warp_reduce_max<kWarpSize>(val);
240     if (lane == 0)
241         shared[warp] = value;
242     __syncthreads();
243     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
244     value = warp_reduce_max<kNumWarps>(value);
245     value = __shfl_sync(0xffffffff, value, 0, 32);
246     return value;
247 }
248
249 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
250 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
251 // 跨 block 求和的常见模式, 适合 N 较大时使用;

```

```

252 // Grid: ((N + 255) / 256, 1, 1)
253 // Block: (256, 1, 1)
254 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
255 template <const int kNumThreads = 256>
256 __global__ void block_reduce_all(float *a, float *y, int N) {
257     int tid = threadIdx.x;
258     int idx = blockIdx.x * kNumThreads + tid;
259     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
260     __shared__ float shared[kNumWarps];
261
262     float val = (idx < N) ? a[idx] : 0.0f;
263     int warp = tid / kWarpSize;
264     int lane = tid % kWarpSize;
265
266     val = warp_reduce_sum<kWarpSize>(val);
267     if (lane == 0)
268         shared[warp] = val;
269     __syncthreads();
270
271     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
272     if (warp == 0)
273         val = warp_reduce_sum<kNumWarps>(val);
274     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
275         atomicAdd(y, val);
276 }
277
278 // ---- Dot Product: y = sum(a[i] * b[i]) ----
279 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
280 // Grid: ((N + 255) / 256, 1, 1)
281 // Block: (256, 1, 1)
282 // source: LeetCUDA/kernels/dot-product/dot_product.cu
283 template <const int kNumThreads = 256>
284 __global__ void dot(float *a, float *b, float *y, int N) {
285     int tid = threadIdx.x;
286     int idx = blockIdx.x * kNumThreads + tid;
287     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
288     __shared__ float shared[kNumWarps];
289
290     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
291     int warp = tid / kWarpSize;
292     int lane = tid % kWarpSize;
293
294     prod = warp_reduce_sum<kWarpSize>(prod);
295     if (lane == 0)
296         shared[warp] = prod;
297     __syncthreads();
298
299     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
300     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
301         prod = warp_reduce_sum<kNumWarps>(prod);
302     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果，避免重复累加
303         atomicAdd(y, prod);
304 }
305
306 // Dot Product + float4
307 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素，float4向量化后每个线程处理4元素
308 // Block: (64, 1, 1), 256/4=64
309 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
310 // source: LeetCUDA/kernels/dot-product/dot_product.cu
311 template <const int kNumThreads = 256 / 4>
312 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
313     int tid = threadIdx.x;
314     int idx = (blockIdx.x * kNumThreads + tid) * 4;

```

```

315 constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
316 __shared__ float shared[kNumWarps];
317
318 float4 reg_a = FLOAT4(a[idx]);
319 float4 reg_b = FLOAT4(b[idx]);
320 float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
321                          reg_a.z * reg_b.z + reg_a.w * reg_b.w)
322                  : 0.0f;
323 int warp = tid / kWarpSize;
324 int lane = tid % kWarpSize;
325
326 prod = warp_reduce_sum<kWarpSize>(prod);
327 if (lane == 0)
328     shared[warp] = prod;
329 __syncthreads();
330
331 prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
332 if (warp == 0)
333     prod = warp_reduce_sum<kNumWarps>(prod);
334 if (tid == 0)
335     atomicAdd(y, prod);
336 }
337
338
339 // =====
340 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
341 // =====
342 // 面试要点:
343 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
344 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
345 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
346
347 // ---- ReLU: y = max(0, x) ----
348 // Grid: ((N + 255) / 256, 1, 1)
349 // Block: (256, 1, 1)
350 // source: LeetCUDA/kernels/relu/relu.cu
351 __global__ void relu(float *x, float *y, int N) {
352     int idx = blockIdx.x * blockDim.x + threadIdx.x;
353     if (idx < N)
354         y[idx] = fmaxf(0.0f, x[idx]);
355 }
356
357 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
358 // block(64)*4(float4)=256 元素/block, 与基础版吞吐相同
359 // Grid: ((N + 255) / 256, 1, 1)
360 // Block: (64, 1, 1)
361 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu_vec4(float *x, float *y, int N) {
364     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
365     if (idx < N) {
366         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
367         float4 reg_y;
368         reg_y.x = fmaxf(0.0f, reg_x.x);
369         reg_y.y = fmaxf(0.0f, reg_x.y);
370         reg_y.z = fmaxf(0.0f, reg_x.z);
371         reg_y.w = fmaxf(0.0f, reg_x.w);
372         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
373     }
374 }
375
376 // ---- Elementwise Add: c = a + b ----
377 // Grid: ((N + 255) / 256, 1, 1)

```

```

378 // Block: (256, 1, 1)
379 // source: LeetCUDA/kernels/elementwise/elementwise.cu
380 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
381     int idx = blockIdx.x * blockDim.x + threadIdx.x;
382     if (idx < N)
383         c[idx] = a[idx] + b[idx];
384 }
385
386 // Elementwise Add + float4 向量化
387 // Grid: ((N + 255) / 256, 1, 1)
388 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
389 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
390 // source: LeetCUDA/kernels/elementwise/elementwise.cu
391 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
392     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
393     if ((idx + 3) < N) {
394         float4 reg_a = FLOAT4(a[idx]);
395         float4 reg_b = FLOAT4(b[idx]);
396         float4 reg_c;
397         reg_c.x = reg_a.x + reg_b.x;
398         reg_c.y = reg_a.y + reg_b.y;
399         reg_c.z = reg_a.z + reg_b.z;
400         reg_c.w = reg_a.w + reg_b.w;
401         FLOAT4(c[idx]) = reg_c;
402     } else if (idx < N) {
403         for (int i = 0; (idx + i) < N; i++) {
404             c[idx + i] = a[idx + i] + b[idx + i];
405         }
406     }
407 }
408
409 // ---- Histogram: y[a[i]]++ ----
410 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
411 // Grid: ((N + 255) / 256, 1, 1)
412 // Block: (256, 1, 1)
413 // source: LeetCUDA/kernels/histogram/histogram.cu
414 __global__ void histogram(int *a, int *y, int N) {
415     int idx = blockIdx.x * blockDim.x + threadIdx.x;
416     if (idx < N)
417         atomicAdd(&(y[a[idx]]), 1);
418 }
419
420 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
421 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
422 //
423 // 面试要点:
424 // - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
425 //   每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
426 // - 数学公式 (5 步推导):
427 //   Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
428 //       L_max = max(LSE_1, LSE_2)
429 //   Step 2 — 还原指数权重 (un-normalized):
430 //       w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
431 //   Step 3 — 归一化为混合比例:
432 //       alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
433 //       满足 alpha + beta = 1
434 //   Step 4 — 逐元素加权合并:
435 //       O = alpha * O_1 + beta * O_2
436 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
437 //   lse[head_idx][token_idx])
438 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
439 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
440 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score

```



```

441 // 为 -inf) 的 LSE 可能为 +inf, 替换后  $\exp(-\text{inf} - L_{\text{max}}) = 0$ ,
442 // 该段权重退化为 0
443 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
444 // pack load → FMA → pack store 一条流水线
445 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
446 //
447 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
448 // kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
449 // threads_per_head = head_size / 4 = 2
450 // total_threads = num_tokens * num_heads * threads_per_head = 8
451 //
452 // global_idx:      0      1      2      3      4      5      6      7
453 // token_head_idx:  0      0      1      1      2      2      3      3
454 // (第几个 (token,head) 对, 行优先)
455 // pack_idx:        0      1      0      1      0      1      0      1
456 // (该 (token,head) 对内第几个 pack)
457 // token_idx:       0      0      0      0      1      1      1      1
458 // (token_head_idx / num_heads, token 变化慢)
459 // head_idx:        0      0      1      1      0      0      1      1
460 // (token_head_idx % num_heads, head 变化快)
461 // pack_offset:     0      4      0      4      0      4      0      4
462 // (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
463 // head_offset:     0      0      8      8      16     16     24     24
464 // (token_idx * num_heads * head_size + head_idx * head_size,
465 // 该 (token,head) 对在 output 展平数组中的起始偏移)
466 //
467 // Grid: ((total_threads + 127) / 128, 1, 1)
468 // Block: (128, 1, 1)
469 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
470 __global__ void merge_attn_states(
471     float *output,           // [num_tokens, num_heads, head_size]
472     const float *prefix_output, // [num_tokens, num_heads, head_size]
473     const float *prefix_lse,   // [num_heads, num_tokens]
474     const float *suffix_output, // [num_tokens, num_heads, head_size]
475     const float *suffix_lse,   // [num_heads, num_tokens]
476     int num_tokens, int num_heads, int head_size) {
477
478     constexpr int kNumThreads = 128;
479     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
480     using pack_t = uint4;
481
482     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
483     const int threads_per_head = head_size / kPackSize;
484     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
485     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
486     const int total_threads = num_tokens * num_heads * threads_per_head;
487
488     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
489     if (global_idx >= total_threads)
490         return;
491
492     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
493     // token_head_idx: 第几个 (token, head) 对, 行优先展平
494     const int token_head_idx = global_idx / threads_per_head;
495     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
496     const int pack_idx = global_idx % threads_per_head;
497
498     // token_head_idx 分解为 (token_idx, head_idx)
499     const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
500     const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
501
502     // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
503     const int pack_offset = pack_idx * kPackSize;

```

```

504 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
505 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
506
507 // 定位到当前 (token, head) 的输出段起始
508 const float *prefix_head = prefix_output + head_offset;
509 const float *suffix_head = suffix_output + head_offset;
510 float *output_head = output + head_offset;
511
512 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
513 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
514 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
515
516 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
517 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
518 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
519
520 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
521 const float max_lse = fmaxf(p_lse, s_lse);
522 p_lse -= max_lse;
523 s_lse -= max_lse;
524
525 // Step 2-3: 指数还原 → 混合比例 alpha, beta
526 const float p_se = expf(p_lse);
527 const float s_se = expf(s_lse);
528 const float out_se = p_se + s_se;
529 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
530 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
531
532 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
533 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
534 if (pack_offset < head_size) {
535     pack_t p_pack =
536         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
537     pack_t s_pack =
538         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
539     pack_t o_pack;
540
541 #pragma unroll
542     for (int i = 0; i < kPackSize; ++i) {
543         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
544         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
545         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
546         reinterpret_cast<float *>(&o_pack)[i] = o_v;
547     }
548
549     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
550 }
551 }
552
553 // =====
554 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
555 // =====
556 // 面试要点:
557 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
558 //     online (1-pass, 增量更新)
559 //   - Online Softmax 是 FlashAttention 的数学基础
560 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
561 //     + variance)
562 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
563
564 // =====
565 // Phase 3a: Softmax — 三级递进 (面试核心考点)
566 // =====

```

```

567 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
568 // 回答线索: naive(溢出) → safe → online
569
570 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
571 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
572 // 问题: x 值很大时 exp(x) 溢出为 inf
573 template <const int kNumThreads = 256>
574 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
575 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
576 // source: LeetCUDA/kernels/softmax/softmax.cu
577 __global__ void softmax_per_token(float *x, float *y, int N) {
578     const int tid = threadIdx.x;
579     const int idx = blockIdx.x * blockDim.x + tid;
580
581     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
582     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
583     if (idx < N)
584         y[idx] = exp_val / exp_sum;
585 }
586
587 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
588 // 面试重点: 为什么 Softmax 需要 Safe?
589 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
590 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
591 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
592 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
593 // Grid: (S, 1, 1)
594 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
595 // source: LeetCUDA/kernels/softmax/softmax.cu
596 template <const int kNumThreads = 256>
597 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
598     const int tid = threadIdx.x;
599     const int idx = blockIdx.x * blockDim.x + tid;
600
601     // Pass 1: block reduce max — 找最大值
602     float val = (idx < N) ? x[idx] : -FLT_MAX;
603     float max_val = block_reduce_max<kNumThreads>(val);
604
605     // Pass 2: exp(x - max) → block reduce sum
606     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
607     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
608
609     if (idx < N)
610         y[idx] = exp_val / exp_sum;
611 }
612
613 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
614 // 面试重点:
615 // 两种使用场景 (公式不同, 但结果等价):
616 // 1) 单元素增量更新 (online update, 处理新元素 x_i):
617 //     m_new = max(m_old, x_i)
618 //     d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
619 // 2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
620 //     m = max(m1, m2) safe softmax用的max值
621 //     d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
622 //     当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
623 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
624 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
625 //
626 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)),
627 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
628 // (m1,d1): max 和 Σexp(x_j - m1), 覆盖集合 S1
629 // (m2,d2): max 和 Σexp(x_k - m2), 覆盖集合 S2

```

```

630 //
631 // 合并公式 (对称, 不依赖"新/旧"概念):
632 //   m = max(m1, m2)
633 //   d = d1*exp(m1-m) + d2*exp(m2-m)   ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
634 //
635 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
636 struct __align__(8) MD {
637     float m; // running max
638     float d; // running denominator (sum of exp(x - max))
639 };
640
641 template <const int kWarpWidth = kWarpSize>
642 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
643     #pragma unroll
644     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
645         MD md2;
646         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
647         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);
648
649         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
650         MD s_m = (md1.m > md2.m) ? md2 : md1;
651
652         // b_m.d 无需 rescale (其基准 max 已是全局 max),
653         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
654         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
655         md1.m = b_m.m;
656     }
657     return md1;
658 }
659
660 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
661 // Grid: (S, 1, 1)
662 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
663 // source: LeetCUDA/kernels/softmax/softmax.cu
664 template <const int kNumThreads = 256>
665 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
666     int tid = threadIdx.x;
667     int idx = blockIdx.x * kNumThreads + threadIdx.x;
668     const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
669     __shared__ MD shared[kNumWarps];
670
671     float val = (idx < N) ? x[idx] : -FLT_MAX;
672     int warp = tid / kWarpSize;
673     int lane = tid % kWarpSize;
674
675     // 初始化: 每个线程持有一个 (max, denom) 对
676     MD md;
677     md.m = idx < N ? val : -FLT_MAX;
678     md.d = idx < N ? 1.0f : 0.0f;
679
680     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
681     md = warp_reduce_md<kWarpSize>(md);
682
683     if (lane == 0)
684         shared[warp] = md;
685     __syncthreads();
686
687     // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
688     md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
689     md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果
690
691     // 用全局 max 和 denom 做最终 softmax
692     float d_inv = __fdividef(1.0f, md.d);

```

```

693 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
694 if (idx < N) {
695     y[idx] = __expf(val - md.m) * d_inv;
696 }
697 }
698
699 // =====
700 // Phase 3b: RMS Normalization (1-pass reduce)
701 // =====
702 // 面试要点:
703 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
704 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
705 //   - Llama 系列使用 RMS Norm
706 //   - grid(N, K/K), block(K): 一行一个 block
707 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
708 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
709 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
710 template <const int kNumThreads = 128>
711 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
712     int tid = threadIdx.x;
713     int idx = blockIdx.x * blockDim.x + threadIdx.x;
714     const float epsilon = 1e-5f;
715
716     __shared__ float s_variance;
717     float value = (idx < N * K) ? x[idx] : 0.0f;
718     float variance = value * value;
719     variance = block_reduce_sum<kNumThreads>(variance);
720     if (tid == 0)
721         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
722     __syncthreads();
723     if (idx < N * K)
724         y[idx] = (value * s_variance) * g;
725 }
726
727 // RMS Norm + float4
728 // Grid: (N, 1, 1)
729 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
730 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
731 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
732 template <const int kNumThreads = 128 / 4>
733 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
734     int tid = threadIdx.x;
735     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
736     const float epsilon = 1e-5f;
737
738     __shared__ float s_variance;
739     float4 reg_x = FLOAT4(x[idx]);
740     float4 variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
741                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
742                                     : 0.0f;
743     variance = block_reduce_sum<kNumThreads>(variance);
744     if (tid == 0)
745         s_variance = rsqrtf(variance / (float)K + epsilon);
746     __syncthreads();
747     float4 reg_y;
748     reg_y.x = reg_x.x * s_variance * g;
749     reg_y.y = reg_x.y * s_variance * g;
750     reg_y.z = reg_x.z * s_variance * g;
751     reg_y.w = reg_x.w * s_variance * g;
752     if (idx < N * K)
753         FLOAT4(y[idx]) = reg_y;
754 }
755

```

```

756 // =====
757 // Phase 3c: Layer Normalization (2-pass reduce)
758 // =====
759 // 面试要点:
760 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
761 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)^2)/K)
762 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
763 // Grid: (N, 1, 1), 一行一个 block
764 // Block: (128, 1, 1), kNumThreads=K=128
765 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
766 template <const int kNumThreads = 128>
767 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
768     int tid = threadIdx.x;
769     int idx = blockIdx.x * blockDim.x + threadIdx.x;
770     const float epsilon = 1e-5f;
771
772     __shared__ float s_mean;
773     __shared__ float s_variance;
774     float value = (idx < N * K) ? x[idx] : 0.0f;
775
776     // Pass 1: compute mean
777     float sum = block_reduce_sum<kNumThreads>(value);
778     if (tid == 0)
779         s_mean = sum / (float)K;
780     __syncthreads(); // 必须等待 s_mean 对所有线程可见
781
782     // Pass 2: compute variance = (x - mean)^2
783     float variance = (value - s_mean) * (value - s_mean);
784     variance = block_reduce_sum<kNumThreads>(variance);
785     if (tid == 0)
786         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
787     __syncthreads(); // 必须等待 s_variance 对所有线程可见
788
789     if (idx < N * K)
790         y[idx] = ((value - s_mean) * s_variance) * g + b;
791 }
792
793 // Layer Norm + float4
794 // Grid: (N, 1, 1)
795 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
796 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
797 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
798 template <const int kNumThreads = 128 / 4>
799 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
800     int tid = threadIdx.x;
801     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
802     const float epsilon = 1e-5f;
803
804     __shared__ float s_mean;
805     __shared__ float s_variance;
806     float4 reg_x = FLOAT4(x[idx]);
807     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
808
809     float sum = block_reduce_sum<kNumThreads>(value);
810     if (tid == 0)
811         s_mean = sum / (float)K;
812     __syncthreads();
813
814     float4 reg_x_hat;
815     reg_x_hat.x = reg_x.x - s_mean;
816     reg_x_hat.y = reg_x.y - s_mean;
817     reg_x_hat.z = reg_x.z - s_mean;
818     reg_x_hat.w = reg_x.w - s_mean;

```

```

819 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
820         reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
821 variance = block_reduce_sum<kNumThreads>(variance);
822 if (tid == 0)
823     s_variance = rsqrtf(variance / (float)K + epsilon);
824 __syncthreads();
825
826 float4 reg_y;
827 reg_y.x = reg_x_hat.x * s_variance * g + b;
828 reg_y.y = reg_x_hat.y * s_variance * g + b;
829 reg_y.z = reg_x_hat.z * s_variance * g + b;
830 reg_y.w = reg_x_hat.w * s_variance * g + b;
831 if (idx < N * K)
832     FLOAT4(y[idx]) = reg_y;
833 }
834
835 // =====
836 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
837 // =====
838 // 面试要点:
839 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
840 //     [x1']   [cos(θ)  -sin(θ)] [x1]
841 //     [x2'] = [sin(θ)   cos(θ)] [x2]
842 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
843 //   - token_pos = idx / N: token 在序列中的位置
844 //   - token_idx = idx % N: token 内的维度索引
845 //   - 输入 [seq_len, hidden_size], 输出同形状
846
847 // Grid: ((seq_len * N + 255) / 256, 1, 1)
848 // Block: (256, 1, 1)
849 // source: LeetCUDA/kernels/rope/rope.cu
850 __global__ void rope(float *x, float *out, int seq_len, int N) {
851     int idx = blockIdx.x * blockDim.x + threadIdx.x;
852     float x1 = x[idx * 2];
853     float x2 = x[idx * 2 + 1];
854     int token_pos = idx / N; // 序列位置
855     int token_idx = idx % N; // 维度索引
856
857     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
858     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
859     float sin_v = sinf(token_pos * exp_v);
860     float cos_v = cosf(token_pos * exp_v);
861
862     // 2D 旋转
863     float out1 = x1 * cos_v - x2 * sin_v;
864     float out2 = x1 * sin_v + x2 * cos_v;
865     out[idx * 2] = out1;
866     out[idx * 2 + 1] = out2;
867 }
868
869 // =====
870 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
871 // =====
872 // 面试要点 (Bank Conflict 专题):
873 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
874 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
875 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
876 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
877 //
878 // 转置的四步演进:
879 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
880 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
881 //   BCF:   smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict

```



```

882 // merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
883 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
884
885 // 每个线程处理 1 个元素, block(16,16)
886 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
887 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
888 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
889 // Block: (16, 16, 1)
890 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
891 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
892     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
893     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
894     if (r < row && c < col) {
895         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
896         y[c * row + r] = x[r * col + c];
897     }
898 }
899
900 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
901 // Block: (16, 16, 1)
902 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
903 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
904 __global__ void mat_transpose_padded(
905     float *x, float *y, const int row, const int col) {
906     const int tx = threadIdx.x;
907     const int ty = threadIdx.y;
908
909     constexpr int TILE = 16;
910     constexpr int PAD = 1;
911     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
912     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
913
914     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
915     const int x_c = blockIdx.x * TILE + tx;
916     const int x_r = blockIdx.y * TILE + ty;
917
918     if (x_r * 4 < row && x_c < col) {
919         // Step 1: 4 rows × 1 col → smem (row-major);
920 #pragma unroll
921         for (int i = 0; i < 4; ++i) {
922             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
923         }
924         __syncthreads();
925
926         // Step 2: Transposed read from smem
927         float4 reg_trans;
928         reg_trans.x = tile[tx * 4 + 0][ty];
929         reg_trans.y = tile[tx * 4 + 1][ty];
930         reg_trans.z = tile[tx * 4 + 2][ty];
931         reg_trans.w = tile[tx * 4 + 3][ty];
932
933         // y 空间坐标: y_r = x 的列, y_c = x 的行
934         const int y_r = blockIdx.x * TILE + ty; // = x col
935         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
936
937         // Coalesced write: y[y_r][y_c .. y_c+3]
938         FLOAT4(y[y_r * row + y_c]) = reg_trans;
939     }
940 }
941
942 // =====
943 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
944 // =====

```



```

945 // 面试要点:
946 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
947 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
948 //   - 不同 K 值对应不同分块策略:
949 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
950 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
951 //       K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
952 //
953 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
954
955 // ---- SGEMV K32: 基础 warp-per-row ----
956 // 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
957 // grid(M/4), 每个 warp 负责一行
958 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
959 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
960 // Block: (32, 4, 1), 每行 1 warp
961 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
962 // source: LeetCUDA/kernels/sgemv/sgemv.cu
963 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
964     int tx = threadIdx.x; // 0~31
965     int ty = threadIdx.y; // 0~3
966     int lane = tx % kWarpSize; // 0~31
967     int m = blockIdx.x * blockDim.y + ty; // 全局行号
968     if (m < M) {
969         float sum = 0.0f;
970         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
971         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
972         const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
973 #pragma unroll
974         for (int w = 0; w < NUM_ITERS; ++w) {
975             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
976             int k = w * kWarpSize + lane;
977             sum += a[m * K + k] * x[k];
978         }
979         sum = warp_reduce_sum<kWarpSize>(sum);
980         // 每个 warp 处理一行, lane 0 写回结果
981         if (lane == 0)
982             y[m] = sum;
983     }
984 }
985
986 // ---- SGEMV K128: float4 向量化 ----
987 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
988 // Grid: ((M + 3) / 4, 1, 1)
989 // Block: (32, 4, 1)
990 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
991 // source: LeetCUDA/kernels/sgemv/sgemv.cu
992 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
993     int tx = threadIdx.x; // 0~31
994     int ty = threadIdx.y; // 0~3
995     int lane = tx % kWarpSize; // 0~31
996     int m = blockDim.y * blockIdx.x + ty;
997
998     if (m < M) {
999         float sum = 0.0f;
1000         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1001         const int NUM_ITERS = ((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
1002 #pragma unroll
1003         for (int w = 0; w < NUM_ITERS; ++w) {
1004             int k = (w * kWarpSize + lane) * 4;
1005             float4 reg_x = FLOAT4(x[k]);
1006             float4 reg_a = FLOAT4(a[m * K + k]);
1007             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +

```

```

1008         reg_a.w * reg_x.w);
1009     }
1010     sum = warp_reduce_sum<kWarpSize>(sum);
1011     if (lane == 0)
1012         y[m] = sum;
1013 }
1014 }
1015
1016 // ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
1017 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1018 // kRowPerWarp=2, kNumLanePerRow=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1019 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1020 // Block: (32, 4, 1)
1021 // 注意: 这一版是面向 K=16 的专用写法; kRowPerWarp=2 时一个 warp 同时处理 2 行
1022 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1023 template <const int kRowPerWarp = 2>
1024 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1025     constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
1026     int tx = threadIdx.x;
1027     int ty = threadIdx.y;
1028     int lane = tx % kWarpSize;
1029     int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
1030     int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
1031     if (m < M) {
1032         float sum = A[m * K + k] * x[k];
1033         // 按照kNumLanePerRow=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1034         sum = warp_reduce_sum<kNumLanePerRow>(sum);
1035         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1036         if (k == 0)
1037             y[m] = sum;
1038     }
1039 }
1040
1041 // =====
1042 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1043 // =====
1044 // 面试要点 (GEMM 优化五层金字塔):
1045 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1046 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1047 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1048 //   load/store 指令数 Level 4 — Tensor Core (MMA
1049 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1050 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1051 //
1052 // 计算密度递进:
1053 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1054 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1055 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1056
1057 // =====
1058 // Phase 7a: SGEMM (非 Tensor Core 路径)
1059 // =====
1060
1061 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1062 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1063 // C = A × B, C[M, N] = A[M, K] × B[K, N]
1064 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1065 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1066 // Block: (32, 32, 1), 1024 线程
1067 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1068 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1069     constexpr int BM = 32; // vec 版: 32×4 = 128
1070     constexpr int BN = 32; // vec 版: 32×4 = 128

```

```

1071 constexpr int BK = 32;
1072 __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1073
1074 int bx = blockIdx.x;
1075 int by = blockIdx.y;
1076 int tx = threadIdx.x;
1077 int tid = threadIdx.y * blockDim.x + tx;
1078
1079 // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1080 int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1081 int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1082 int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1083 int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1084 int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1085 int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1086
1087 float sum = 0.f; // 遍历完整的K, slice K;
1088 for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1089     int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1090     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1091     s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1092     int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1093     int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1094     s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1095     __syncthreads(); // 确保整个 smem tile 加载完毕
1096
1097 #pragma unroll
1098     for (int k = 0; k < BK; ++k) {
1099         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1100         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1101         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1102     }
1103     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1104 }
1105 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1106 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1107 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1108 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1109 }
1110
1111 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1112 // 在 Level 1 基础上引入两层优化:
1113 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1114 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1115 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1116 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1117 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1118 // 1024 x 16 = 16384 = 128 x 128 ✓
1119 //
1120 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1121 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1122 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1123 //
1124 // 加载映射 (每线程 4 个元素, float4):
1125 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1126 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1127 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1128 //
1129 // △ Bank Conflict 提示 (面试加分点):
1130 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1131 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1132 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1133 //

```

```

1134 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1135 // Block: (32, 32, 1), 1024 线程
1136 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1137 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1138 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1139     constexpr int BM = 128;
1140     constexpr int BN = 128;
1141     constexpr int BK = 32;
1142     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1143     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1144
1145     int bx = blockIdx.x;
1146     int by = blockIdx.y;
1147     int tx = threadIdx.x;
1148     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1149
1150     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1151     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1152     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1153     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1154     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1155     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1156
1157     int load_gmem_a_m = by * BM + load_smem_a_m;
1158     int load_gmem_b_n = bx * BN + load_smem_b_n;
1159
1160     // 4x4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1161     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1162     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1163     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1164     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1165     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1166
1167     float sum[4][4] = {0.f};
1168     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1169         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1170         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1171         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1172         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1173         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1174         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1175         __syncthreads();
1176
1177     #pragma unroll
1178     for (int k = 0; k < BK; ++k) {
1179         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
1180         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1181                             s_a[comp_smem_a_m_base + 1][k],
1182                             s_a[comp_smem_a_m_base + 2][k],
1183                             s_a[comp_smem_a_m_base + 3][k]};
1184         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1185                             s_b[k][comp_smem_b_n_base + 1],
1186                             s_b[k][comp_smem_b_n_base + 2],
1187                             s_b[k][comp_smem_b_n_base + 3]};
1188
1189     #pragma unroll
1190     for (int i = 0; i < 4; ++i) {
1191     #pragma unroll
1192         for (int j = 0; j < 4; ++j) {
1193             sum[i][j] += a_vals[i] * b_vals[j];
1194         }
1195     }
1196     __syncthreads();

```

```

1197 }
1198
1199 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1200 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1201 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1202 #pragma unroll
1203 for (int i = 0; i < 4; ++i) {
1204     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1205     float4 reg_c;
1206     reg_c.x = sum[i][0];
1207     reg_c.y = sum[i][1];
1208     reg_c.z = sum[i][2];
1209     reg_c.w = sum[i][3];
1210     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1211 }
1212 }
1213
1214 // =====
1215 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1216 // =====
1217 // 面试要点:
1218 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1219 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1220 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1221 // 寄存器)
1222 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1223 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1224 //
1225 // ★ TN 布局详解 (面试高频考点):
1226 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1227 // BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1228 // N = Normal (列优先, BLAS 原生格式)
1229 // T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1230 // 第一个字母 → A 的 op, 第二个字母 → B 的 op
1231 // 所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1232 // BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1233 // 视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1234 // 调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1235 // T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1236 // N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1237 //
1238 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1239 // T = row-major (行优先, 对 BLAS 来说是 transposed)
1240 // N = col-major (列优先, BLAS native = normal)
1241 //
1242 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
1243 // - A: row-major [M, K], B: row-major [K, N]
1244 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1245 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1246 //
1247 // TN 布局: C[MxN] = A[MxK] × B[KxN], B 以 B^T=[NxK] row-major 存储 (A 行优先, B 列优先)
1248 // - A [MxK]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1249 // - B^T [NxK]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1250 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1251 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1252 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1253 //
1254 // WGMMMA (Warp Group MMA, Hopper+):
1255 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1256 // - m64n128k16: 一次处理 64x128x16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1257 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1258 // threads) 做计算
1259 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址

```

```

1260
1261 // =====
1262 // Phase 7b-1: MMA PTX 宏定义
1263 // =====
1264
1265 // ---- gmem → smem: cp.async ----
1266 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1267 //   - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1268 //   - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1269 //     N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1270 //   - wait_all: 等价于 commit_group + wait_group 0。
1271 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1272 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1273 #define CP_ASYNC_WAIT_GROUP(n) \
1274     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1275
1276 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1277 #define CP_ASYNC_CG(dst, src, bytes) \
1278     asm volatile( \
1279         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1280         "l"(src), "n"(bytes))
1281
1282 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1283 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1284 // 每次加载 8x8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1285 // aligned: 要求 128-bit 对齐, trans: 转置加载
1286 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1287     asm volatile( \
1288         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1289         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1290         : "r"(addr))
1291
1292 #define LDMATRIX_X2(R0, R1, addr) \
1293     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1294         : "=r"(R0), "=r"(R1) \
1295         : "r"(addr))
1296
1297 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1298 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1299 #define LDMATRIX_X2_T(R0, R1, addr) \
1300     asm volatile( \
1301         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1302         : "=r"(R0), "=r"(R1) \
1303         : "r"(addr))
1304
1305 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1306 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1307 // row.col: A 是 row-major, B 是 col-major
1308 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1309 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1310 // C 矩阵大小 16x8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1311 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1312     asm volatile( \
1313         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1314         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1315         : "=r"(RD0), "=r"(RD1) \
1316         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1317         "r"(RC1))
1318
1319 // ---- MMA 辅助函数 ----
1320 // div_ceil: 整数除法向上取整
1321 #define HOST_DEVICE_INLINE __device__ __host__ inline
1322 HOST_DEVICE_INLINE int div_ceil(int a, int b) {

```



```

1323 return (a % b != 0) ? (a / b + 1) : (a / b);
1324 }
1325
1326 // =====
1327 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1328 // =====
1329 // 面试重点 — Tile Hierarchy:
1330 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1331 //   MMA Tile (more warps): 2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
1332 //   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
1333 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1334 //           → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1335 //
1336 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1337 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1338 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1339 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1340 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1341 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1342 //
1343 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1344 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1345 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1346 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1347 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1348 //
1349 // ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
1350 //   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
1351 //   (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
1352 //   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
1353 //
1354 //   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
1355 //   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
1356 //
1357 //       N / bx → 0          1          2          3          ...
1358 //       M ↓
1359 //       by = 0   | SM0 | SM1 | SM2 | SM3 | ...
1360 //                | A 0,B0 | A 0,B1 | A 0,B2 | A 0,B3 |
1361 //                +-----+
1362 //
1363 //   同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
1364 //   推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
1365 //   L2 更容易被其他 B tile 挤占。
1366 //
1367 //   Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
1368 //   下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:
1369 //
1370 //       N / local x → 0          1
1371 //       M ↓
1372 //       by = 0   | SM0 | SM1 | → A 0; B^T 0, B^T 1
1373 //                +-----+
1374 //       by = 1   | SM2 | SM3 | → A 1; B^T 0, B^T 1
1375 //                +-----+
1376 //
1377 //   对于这个 2x2 窗口: 同一行横向复用 A (SM0/SM1、SM2/SM3), 同一列纵向复用 B
1378 //   (SM0/SM2、SM1/SM3)。真实 grid_x 不必等于 2, 但缩小它会缩短再次访问相同 bx 的距离。
1379 //   此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会, 不保证 CTA 的 z 顺序、
1380 //   缓存驻留或 L2 hit。
1381 //
1382 //   Launch (kBlockSwizzle=false, 当前默认路径):
1383 //       grid = (ceil(N / BN), ceil(M / BM), 1), blockIdx.x 直接就是 N-tile 编号 bx。
1384 //   Launch (kBlockSwizzle=true, 需由 launch 端显式构造 3D grid):
1385 //       tiles_n = ceil(N / BN), S = ceil(N / 2048)

```

```

1386 // grid = (ceil(tiles_n / S), ceil(M / BM), S)
1387 // bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
1388 // 例如 N=8192、BN=128: tiles_n=64, S=4, grid.x=16; z=0/1/2/3 分别覆盖
1389 // bx=0..15/16..31/32..47/48..63, 即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
1390 // grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
1391 // 对齐; ceil 只保证逻辑 tile 编号不遗漏, 并不使部分尾 tile 变为安全。
1392 // Block: (256, 1, 1), 8 warps
1393 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1394 // =====
1395 template <const int kMmaM = 16, // MMA atom M dim (m16n8k16)
1396         const int kMmaN = 8, // MMA atom N dim
1397         const int kMmaK = 16, // MMA atom K dim, also BK tile = kMmaK
1398         const int kMmaTileM = 2, // warps along M, 2 → warp tile M = 32
1399         const int kMmaTileN = 4, // warps along N, 4 → warp tile N = 32
1400         const int kValTileM = 4, // value-repeat along M, BM = 16*2*4 = 128
1401         const int kValTileN = 4, // value-repeat along N, BN = 8*4*4 = 128
1402         const int kStages = 3, // cp.async pipeline depth
1403         const int kBlockSwizzle = 0> // 1 enables 3D grid swizzle for L2 locality
1404 __global__ void __launch_bounds__(256)
1405     hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1406     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1407     // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
1408     // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
1409     const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1410     const int by = blockIdx.y;
1411     constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1412     constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1413     constexpr int BK = kMmaK; // 16
1414
1415     // Dynamic shared memory: kStages 个 stage 的 A 和 B
1416     // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1417     // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1418     // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1419     // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1420     extern __shared__ half smem[];
1421     half *s_a = smem;
1422     half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1423     constexpr int s_a_stage_offset = BM * BK; // 128*16
1424     constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major
1425     const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1426     const int warp_id = tid / kWarpSize; // 0~7
1427     const int lane_id = tid % kWarpSize; // 0~31
1428     // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1429     // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1430     const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1431     const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1432
1433     // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1434     // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1435     int load_smem_a_m = tid / 2; // 0~127
1436     int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1437     int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1438     int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1439     int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1440     int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1441     if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1442         return;
1443
1444     // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32]。
1445     // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1446     // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1447     // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1448     uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0

```



```

1449
1450 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1451 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1452 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1453
1454 // 预加载前 (kStages-1) 个 stage
1455 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1456 #pragma unroll
1457 for (int k = 0; k < (kStages - 1); ++k) {
1458     int load_gmem_a_k = k * BK + load_smem_a_k;
1459     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1460     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1461         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1462     );
1463     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1464
1465     int load_gmem_b_k = k * BK + load_smem_b_k;
1466     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1467     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1468     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1469         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1470     );
1471     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1472
1473     CP_ASYNC.COMMIT_GROUP();
1474 }
1475
1476 CP_ASYNC.WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1477 __syncthreads();
1478
1479 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1480 const int NUM_K_TILES = div_ceil(K, BK);
1481 #pragma unroll
1482 for (int k = 0; k < NUM_K_TILES; ++k) {
1483     int smem_sel = k % kStages; // 计算 tile k 的 stage
1484     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1485
1486     // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
1487     // 加载了(kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1488     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1489     // (row-major, 内维连续的是 K)
1490     if (k + kStages - 1 < NUM_K_TILES) {
1491         int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1492         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1493         int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1494         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1495
1496         uint32_t load_smem_a_ptr =
1497             (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1498                 load_smem_a_m * BK + load_smem_a_k) *
1499                 sizeof(half));
1500         CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1501
1502         uint32_t load_smem_b_ptr =
1503             (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1504                 load_smem_b_n * BK + load_smem_b_k) *
1505                 sizeof(half));
1506         CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1507         CP_ASYNC.COMMIT_GROUP();
1508     }
1509
1510     // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1511     // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中

```

```

1512 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1513 uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1514 uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1515
1516 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1517 #pragma unroll
1518 for (int i = 0; i < kValTileM; ++i) {
1519     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1520     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1521     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1522     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1523     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1524     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1525     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1526     uint32_t lane_smem_a_ptr =
1527         (smem_a_base_ptr +
1528          (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1529           sizeof(half));
1530     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1531 }
1532
1533 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1534 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1535 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1536 #pragma unroll
1537 for (int j = 0; j < kValTileN; ++j) {
1538     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1539     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1540     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1541     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1542     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1543     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1544     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1545     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1546     uint32_t lane_smem_b_ptr =
1547         (smem_b_base_ptr +
1548          (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1549           sizeof(half));
1550     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1551 }
1552
1553 // MMA compute: 每个Warp(MMA) 在M方向重复VAL_TILE_M(4)次, 在N方向重复VAL_TILE_N(4)次
1554 #pragma unroll
1555 for (int i = 0; i < kValTileM; ++i) {
1556     #pragma unroll
1557     for (int j = 0; j < kValTileN; ++j) {
1558         HMMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1559                  RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1560                  RB[j][0], RB[j][1], // B fragment
1561                  RC[i][j][0], RC[i][j][1]);
1562     }
1563 }
1564
1565 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1566 if (k + kStages - 1 < NUM_K_TILES) {
1567     CP_ASYNC_WAIT_GROUP(kStages - 2);
1568 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1569     CP_ASYNC_WAIT_GROUP(0);
1570 }
1571 __syncthreads();
1572 }
1573
1574 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)

```

```

1575 {
1576     for (int i = 0; i < kValTileM; ++i) {
1577         // RC[kValTileM][kValTileN][2] 中 RC[...] [...] [2] 中保存了2个 uint32
1578         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...] [...] [0]
1579         // 和RC[...] [...] [1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1580         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1581         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1582         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1583         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1584         // =====
1585         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1586         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1587         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1588         //
1589         //      cols: c0-1    c2-3    c4-5    c6-7
1590         //  r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1591         //  r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1592         //  r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1593         //  ...
1594         //  r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1595         //  r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1596         //  r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1597         //  r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1598         //  ...
1599         //  r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1600         //
1601         // Within a 4-lane group (e.g. T0-T3 for row 0):
1602         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1603         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1604         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1605         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1606         // =====
1607     #pragma unroll
1608     for (int j = 0; j < kValTileN; ++j) {
1609         RC0[j][0] = RC[i][j][0];
1610         RC1[j][0] = RC[i][j][1];
1611         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1612         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1613         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1614         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1615         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1616         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1617     }
1618     // 每 4 个 lane 中只有 lane 0 做 128-bit store
1619     // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1620     //
1621     // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1622     // |-----|-----|-----|-----|
1623     // | 0~3   | 0       | row 0     | row 8     |
1624     // | 4~7   | 1       | row 1     | row 9     |
1625     // | 8~11  | 2       | row 2     | row 10    |
1626     // | 12~15 | 3       | row 3     | row 11    |
1627     // | 16~19 | 4       | row 4     | row 12    |
1628     // | 20~23 | 5       | row 5     | row 13    |
1629     // | 24~27 | 6       | row 6     | row 14    |
1630     // | 28~31 | 7       | row 7     | row 15    |
1631     //
1632     if (lane_id % 4 == 0) {
1633         // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1634         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1635         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1636     #pragma unroll
1637     for (int j = 0; j < kValTileN; ++j) {

```

```

1638 // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1639 int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1640 int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1641 int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1642 int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
1643 // 128-bit store: 一次写入 8 个 half
1644 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1645     *reinterpret_cast<float4*>(&RC0[j][0]);
1646 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1647     *reinterpret_cast<float4*>(&RC1[j][0]);
1648 }
1649 }
1650 }
1651 }
1652 }
1653
1654 // =====
1655 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1656 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1657 // =====
1658 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1659 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1660 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1661 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1662 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1663 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1664 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1665 //   rows 8~11: repeat rows 0~3 pattern
1666 //   rows 12~15: repeat rows 4~7 pattern
1667 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
1668 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1669 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1670 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1671 //
1672 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1673 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1674
1675 // ---- Swizzle 辅助函数 ----
1676
1677 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1678 // i: row index; j: col index.
1679 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1680 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1681 // 用位运算展开 (kStep=8) : (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1682 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1683 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1684 template <const int kColStride = 16, const int kStep = 8>
1685 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1686     // for col_stride > 16, we have to permute it using col major ZigZag order.
1687     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1688     static_assert(kColStride <= 16, "kColStride must <= 16");
1689     // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1690     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1691     static_assert(kColStride % kStep == 0,
1692         "kColStride must be multiple of kStep.");
1693     if constexpr (kStep == 8) {
1694         // j >> 3: 表示8个half(=16 bytes)数据为一个chunk; i >> 2: 表示4行为一组
1695         // kColStride >> 3: 按照kStep=8计算chunk_idx, kColStride=16 → {0, 1}
1696         // 最后的 << 3: 将chunk_idx转换为实际的列偏移量 (8个half/chunk), {0, 8}
1697         return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1698     } else {
1699         static_assert(kStep == 4);
1700         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;

```

```

1701     }
1702 }
1703
1704 // swizzle_A: A 矩阵专用封装 (kMmaAtomK=16, kStep=8) 。
1705 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1706 // 8个half=16 bytes=128 bits=4 banks (32 bits/bank) 组成一个phase,
1707 // 触发一次合并的memory transaction. 这里的col=16的swizzle, 相当于
1708 // TMA中的SWIZZLE_32B pattern.
1709 // -----
1710 // -col 0~16, step 8--
1711 // -----
1712 // | row 0 | (0, 8) |
1713 // | row 1 | (0, 8) |
1714 // | row 2 | (0, 8) |
1715 // | row 3 | (0, 8) |
1716 // -----
1717 // | row 4 | (8, 0) |
1718 // | row 5 | (8, 0) |
1719 // | row 6 | (8, 0) |
1720 // | row 7 | (8, 0) |
1721 // -----
1722 // | row 8 | (0, 8) |
1723 // | row 9 | (0, 8) |
1724 // | row 10 | (0, 8) |
1725 // | row 11 | (0, 8) |
1726 // -----
1727 // | row 12 | (8, 0) |
1728 // | row 13 | (8, 0) |
1729 // | row 14 | (8, 0) |
1730 // | row 15 | (8, 0) |
1731 // -----
1732 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1733 template <const int kMmaAtomK = 16>
1734 static __device__ __forceinline__ int swizzle_A(int i, int j) {
1735     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1736 }
1737
1738 // swizzle_B: B 矩阵专用封装 (与 A 相同的 pattern) 。
1739 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1740 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8) 。
1741 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1742 template <const int kMmaAtomK = 16>
1743 static __device__ __forceinline__ int swizzle_B(int i, int j) {
1744     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1745 }
1746
1747 // =====
1748 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1749 //           + Register Double Buffering (kValTileK=2, BK=32 统一 tile)
1750 // =====
1751 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1752 // - kValTileK=2: 每个 BK tile 包含 2 个 kMmaK slice (BK = kMmaK * kValTileK = 32)
1753 // - BK=32 统一 tile: kMmaK=0 和 kMmaK=1 数据连续存放在同一个 BK=32 宽的 smem tile 中
1754 // - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换
1755 // - ldmatrix 与 MMA 计算重叠: 加载 k_step=1 的同时用另一组寄存器做 k_step=0 计算
1756 //
1757 // ★ smem 布局:
1758 //   s_a: [stage0][stage1][stage2], 每个 stage = BM × BK = 128 × 32
1759 //   s_b: 紧接 s_a 之后
1760 //   每个 stage 内 k_step=0 列偏移 0, k_step=1 列偏移 +kMmaK(=16)
1761 //
1762 // ★ 寄存器双缓冲时间线 (每 k 迭代内):
1763 //   Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)

```

```

1764 // Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1765 // Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1766 // Step 4: adaptive wait + __syncthreads
1767 // Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1768 //
1769 // 与 dsmem 版本的对比:
1770 // - dsmem: smem 分两个区域存放 kMmaK=0 和 kMmaK=1, smem 翻倍
1771 // - BK=32: kMmaK=0/1 连续存同一 tile 内, smem 不变, gmem 索引用 k*BK 直接定位
1772 //
1773 // 参考: LeetCUDA/kernels/hgemmm/mma/swizzle/hgemmm_mma_stage_tn_swizzle_x2.cu
1774 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1775 // Block: (256, 1, 1), 8 warps
1776 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1777           const int kMmaN = 8,           // MMA atom N dim
1778           const int kMmaK = 16,          // MMA atom K dim
1779           const int kMmaTileM = 2,        // warps along M, 2 → warp tile M = 32
1780           const int kMmaTileN = 4,        // warps along N, 4 → warp tile N = 32
1781           const int kValTileM = 4,        // value-repeat along M, BM = 16*2*4 = 128
1782           const int kValTileN = 4,        // value-repeat along N, BN = 8*4*4 = 128
1783           const int kValTileK = 2,        // MMA_K slices per BK tile, BK = kMmaK*2 = 32
1784           const int kStages = 3,          // cp.async pipeline depth
1785           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
1786 __global__ void __launch_bounds__(256)
1787   hgemmm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1788   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1789   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1790   const int bx = ((int)kBlockSwizzle) * blockDim.z * gridDim.x + blockIdx.x;
1791   const int by = blockIdx.y;
1792   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1793   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1794   constexpr int BK = kMmaK * kValTileK;             // 16*2=32
1795
1796   // kStages stages, each with BM×BK for A, BN×BK for B
1797   // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK=32 宽 tile 中
1798   extern __shared__ half smem[];
1799   half *s_a = smem;
1800   half *s_b = smem + kStages * BM * BK;
1801   constexpr int s_a_stage_offset = BM * BK;
1802   constexpr int s_b_stage_offset = BN * BK;
1803
1804   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1805   const int warp_id = tid / kWarpSize;
1806   const int lane_id = tid % kWarpSize;
1807   const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1808   const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1809
1810   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1811   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1812   // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 虽然BK=16*2=32, 在后续的kValTileK循环中
1813   // 会加上 k_step*kMmaK=0/16, 最终得到 smem 中的列偏移 0/8/16/24, 正好覆盖 BK=32
1814   int load_smem_a_m = tid / 2;           // 0~127
1815   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1816   int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1817   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1818   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1819   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1820   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1821     return;
1822
1823   // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1824   // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1825   // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1826   // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。

```



```

1827     uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1828
1829     uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1830     uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1831
1832     // Prefetch (kStages-1) stages: load all k_step slices for each early stage
1833     #pragma unroll
1834     for (int k = 0; k < (kStages - 1); ++k) {
1835         #pragma unroll
1836         for (int k_step = 0; k_step < kValTileK; ++k_step) {
1837             int load_gmem_a_k = k * BK + k_step * kMmaK + load_smem_a_k;
1838             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1839             int load_gmem_b_k = k * BK + k_step * kMmaK + load_smem_b_k;
1840             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1841
1842             uint32_t load_smem_a_ptr =
1843                 (smem_a_base_ptr +
1844                  (k * s_a_stage_offset + load_smem_a_m * BK +
1845                   k_step * kMmaK +
1846                    swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1847                   sizeof(half));
1848             CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1849
1850             uint32_t load_smem_b_ptr =
1851                 (smem_b_base_ptr +
1852                  (k * s_b_stage_offset + load_smem_b_n * BK +
1853                   k_step * kMmaK +
1854                    swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1855                   sizeof(half));
1856             CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1857         }
1858         CP_ASYNC.COMMIT_GROUP();
1859     }
1860
1861     CP_ASYNC.WAIT_GROUP(kStages - 2);
1862     __syncthreads();
1863
1864     // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1865     uint32_t RA[2][kValTileM][4];
1866     uint32_t RB[2][kValTileN][2];
1867     int reg_st_idx = 0; // write target for ldmatrix
1868     int reg_ld_idx = 1; // read source for MMA
1869
1870     // Initial ldmatrix: load stage 0, k_step=0 (kMmaK=0 列) → reg[0]
1871     #pragma unroll
1872     for (int i = 0; i < kValTileM; ++i) {
1873         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1874         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1875         int lane_smem_a_k = (lane_id / 16) * 8;
1876         uint32_t lane_smem_a_ptr =
1877             (smem_a_base_ptr +
1878              (0 * s_a_stage_offset + lane_smem_a_m * BK +
1879               swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1880              sizeof(half));
1881         LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1882                    RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1883                    lane_smem_a_ptr);
1884     }
1885     #pragma unroll
1886     for (int j = 0; j < kValTileN; ++j) {
1887         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1888         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1889         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;

```



```

1890     uint32_t lane_smem_b_ptr =
1891         (smem_b_base_ptr +
1892          (0 * s_b_stage_offset + lane_smem_b_n * BK +
1893           swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1894            sizeof(half));
1895     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1896                 lane_smem_b_ptr);
1897 }
1898
1899 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
1900 // BK = kMmaK * kValTileK = 32, 每个 k 迭代覆盖 BK=32 个 K 元素
1901 const int NUM_K_TILES = div_ceil(K, BK);
1902 #pragma unroll
1903 for (int k = 0; k < NUM_K_TILES; ++k) {
1904     int smem_sel = k % kStages;
1905     int smem_sel_next = (k + kStages - 1) % kStages;
1906
1907     // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
1908     if (k + kStages - 1 < NUM_K_TILES) {
1909 #pragma unroll
1910         for (int k_step = 0; k_step < kValTileK; ++k_step) {
1911             int load_gmem_a_k =
1912                 (k + kStages - 1) * BK + k_step * kMmaK + load_smem_a_k;
1913             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1914             int load_gmem_b_k =
1915                 (k + kStages - 1) * BK + k_step * kMmaK + load_smem_b_k;
1916             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1917
1918             uint32_t load_smem_a_ptr =
1919                 (smem_a_base_ptr +
1920                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1921                   k_step * kMmaK +
1922                    swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1923                     sizeof(half));
1924             CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1925
1926             uint32_t load_smem_b_ptr =
1927                 (smem_b_base_ptr +
1928                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1929                   k_step * kMmaK +
1930                    swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1931                     sizeof(half));
1932             CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1933         }
1934         CP_ASYNC_COMMIT_GROUP();
1935     }
1936
1937     // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
1938     // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
1939     // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
1940
1941 #pragma unroll
1942     for (int k_step = 0; k_step < kValTileK; ++k_step) {
1943         reg_st_idx ^= 1; // 0 -> 1; 1 -> 0
1944         reg_ld_idx ^= 1; // 1 -> 0; 0 -> 1
1945
1946         if (k_step + 1 < kValTileK) {
1947             int smem_k_offset = (k_step + 1) * kMmaK;
1948 #pragma unroll
1949             for (int i = 0; i < kValTileM; ++i) {
1950                 int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1951                 int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1952                 int lane_smem_a_k = (lane_id / 16) * 8;

```

```

1953     uint32_t lane_smem_a_ptr =
1954         (smem_a_base_ptr +
1955         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1956         smem_k_offset +
1957         swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1958         sizeof(half));
1959     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1960         RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1961         lane_smem_a_ptr);
1962 }
1963 #pragma unroll
1964 for (int j = 0; j < kValTileN; ++j) {
1965     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1966     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1967     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1968     uint32_t lane_smem_b_ptr =
1969         (smem_b_base_ptr +
1970         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1971         smem_k_offset +
1972         swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1973         sizeof(half));
1974     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1975         lane_smem_b_ptr);
1976 }
1977 }
1978
1979 #pragma unroll
1980 for (int i = 0; i < kValTileM; ++i) {
1981     #pragma unroll
1982     for (int j = 0; j < kValTileN; ++j) {
1983         HMMA16816(RC[i][j][0], RC[i][j][1],
1984             RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
1985             RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
1986             RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
1987             RC[i][j][0], RC[i][j][1]);
1988     }
1989 }
1990 }
1991
1992 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
1993 if (k + kStages - 1 < NUM_K_TILES) {
1994     CP_ASYNC_WAIT_GROUP(kStages - 2);
1995 } else {
1996     CP_ASYNC_WAIT_GROUP(0);
1997 }
1998 __syncthreads();
1999
2000 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
2001 if (k + 1 < NUM_K_TILES) {
2002     smem_sel = (k + 1) % kStages; // old smem_sel + 1
2003     #pragma unroll
2004     for (int i = 0; i < kValTileM; ++i) {
2005         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2006         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2007         int lane_smem_a_k = (lane_id / 16) * 8;
2008         uint32_t lane_smem_a_ptr =
2009             (smem_a_base_ptr +
2010             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2011             swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2012             sizeof(half));
2013         LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2014             RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2015             lane_smem_a_ptr);

```

```

2016     }
2017 #pragma unroll
2018     for (int j = 0; j < kValTileN; ++j) {
2019         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2020         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2021         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2022         uint32_t lane_smem_b_ptr =
2023             (smem_b_base_ptr +
2024              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2025               swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2026               sizeof(half));
2027         LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2028                     lane_smem_b_ptr);
2029     }
2030 }
2031 }
2032
2033 // Epilogue: 复用 RA 寄存器做 warp shuffle → 128-bit collective store
2034 for (int i = 0; i < kValTileM; ++i) {
2035 #pragma unroll
2036     for (int j = 0; j < kValTileN; ++j) {
2037         RA[0][j][0] = RC[i][j][0];
2038         RA[1][j][0] = RC[i][j][1];
2039         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
2040         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
2041         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
2042         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
2043         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
2044         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
2045     }
2046     if (lane_id % 4 == 0) {
2047         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2048         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2049 #pragma unroll
2050         for (int j = 0; j < kValTileN; ++j) {
2051             int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2052             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2053             int store_gmem_c_addr_0 =
2054                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2055             int store_gmem_c_addr_1 =
2056                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2057             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2058                 *reinterpret_cast<float4*>(&RA[0][j][0]);
2059             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2060                 *reinterpret_cast<float4*>(&RA[1][j][0]);
2061         }
2062     }
2063 }
2064 }
2065
2066 // =====
2067 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2068 // =====
2069 // 面试要点 (CuTe vs 手写 MMA PTX) :
2070 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2071 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2072 // - 与手写 MMA PTX (Phase 7b) 的对比:
2073 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
2074 //   - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2075 //     cute::copy / cute::gemm 自动展开为高效指令序列
2076 // - 核心抽象 ("CuTe 五要素") :
2077 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2078 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type

```

```

2079 // 3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2080 // 4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2081 // 5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2082 //
2083 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型):
2084 // MMA Atom (SM80_16x8x16_F16F16F16_TN)
2085 // → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2086 // → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2087 // → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2088 //
2089 // 调参口诀 (面试速记):
2090 // BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2091 // Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2092 // 重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2093 // 1. 连续的 2^M 个元素组成二维空间中的一个基本元素;
2094 // 2. 连续的 2^S 个基本元素组成一行;
2095 // 3. 二维空间包含 2^B 行;
2096 // 4. 对二维坐标做列置换: icol' = irow ^ icol;
2097 // 5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2098 // 因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2099 // CuTe 的位级实现等价于:
2100 // offset' = offset ^ ((offset & YYY) >> S),
2101 // YYY = ((1 << B) - 1) << (M + S)。
2102 // 对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2103 // 二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2104 // 一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2105 // kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2106 // EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2107 // 与逻辑 tile 组织; MMA atom 越多, 需要的线程数就越多
2108 //
2109 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2110 // - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2111 // - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2112 // - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2113 //
2114 // ★ 本实现的 Tile 配置:
2115 // - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2116 // - MMA atom: SM80_16x8x16_F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2117 // - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2118 // BN=256 不是由“8x2x16”直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2119 // - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2120 // 的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2121 // - Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)。
2122 // - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2123 //
2124 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2125 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2126 // =====
2127
2128 #if defined(NOTES_V2_ENABLE_CUTE)
2129
2130 #include <cute/tensor.hpp>
2131
2132 // =====
2133 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2134 // =====
2135 // TN 布局: C[MxN] = A[MxK] × B^T[NxK]
2136 // - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [KxN])
2137 // - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2138 //
2139 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽, 不是一条完整计算链):
2140 // 1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async), 填满 stage。
2141 // 2. 主循环 over K tiles:

```

```

2142 // a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
2143 // b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
2144 // c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的 G→S;
2145 // d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2146 // 3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store) 。
2147 //
2148 // 面试对比 — 手写 MMA vs CuTe 代码量:
2149 // 手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2150 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2151 template <typename T,                                // element type (half)
2152          int BM, int BN, int BK,                      // block tile: BM=128, BN=256, BK=32
2153          int kStage,                                  // cp.async pipeline depth (2)
2154          typename TiledMMA,                          // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
2155          typename G2SCopyA,                          // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2156          typename G2SCopyB,                          // G→S B: same as G2SCopyA
2157          typename SmemLayoutA,                       // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2158          typename SmemLayoutB,                       // smem B: same atom → (BN,BK,kStage)
2159          typename SmemLayoutC,                       // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2160          typename S2RCopyAtomA,                      // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
2161          typename S2RCopyAtomB,                      // S→R B: same as S2RCopyAtomA
2162          typename R2SCopyAtomC,                      // R→S C: UniversalCopy<int> (32-bit store)
2163          typename S2GCopyAtomC,                      // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2164          typename S2GCopyC,                          // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2165          const int BlockSwizzle = 0>                // 1 enables 3D grid swizzle for L2 locality
2166 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2167                                           int n, int k) {
2168     using namespace cute;
2169     static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
2170
2171     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
2172     extern __shared__ T shm_data[];
2173     T *Ashm = shm_data;
2174     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2175
2176     int idx = threadIdx.x;
2177     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2178     int ix = ((int)BlockSwizzle) * blockDim.z * gridDim.x + blockIdx.x;
2179     int iy = blockIdx.y; // M 方向 block 索引
2180     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2181     // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2182     if (iy * BM >= m || ix * BN >= n) return;
2183
2184     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2185     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2186     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2187                             make_stride(k, Int<1>{}));
2187     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2188                             make_stride(k, Int<1>{}));
2189     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2190                             make_stride(n, Int<1>{}));
2191
2192     // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2193     // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2194     // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2195     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2196     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2197     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2198
2199     // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2200     auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2201     auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2202
2203     // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment

```

```

2205 TiledMMA tiled_mma;
2206 auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2207 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2208 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2209 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2210 clear(tCrD); // 累加器清零
2211
2212 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2213 G2SCopyA g2s_tiled_copy_a;
2214 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2215 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2216 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2217
2218 G2SCopyB g2s_tiled_copy_b;
2219 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2220 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2221 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2222
2223 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2224 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2225 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2226 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2227 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2228 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2229
2230 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2231 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2232 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2233 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2234
2235 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2236 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2237 #pragma unroll
2238 for (int istage = 0; istage < kStage - 1; ++istage) {
2239     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2240               tAsA_copy(_, _, _, istage));
2241     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2242               tBsB_copy(_, _, _, istage));
2243     cp_async_fence();
2244     ++itile_to_read;
2245     ++ismem_write;
2246 }
2247 cp_async_wait<kStage - 2>();
2248 __syncthreads();
2249
2250 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2251 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2252 // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2253 int num_k_tiles = k / BK;
2254 // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2255 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2256 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2257
2258 #pragma unroll 1
2259 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2260     // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode)。
2261     //
2262     // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2263     // tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2264     // - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix),
2265     // 所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2266     // - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2267     // 同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)

```



```

2268 // 在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2269 // 无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2270 //     for (i=0; i<kValTileM; ++i)
2271 //         for (j=0; j<kValTileN; ++j)
2272 //             HMMMA16816(RC[i][j][0], RC[i][j][1], ...);
2273 //
2274 // CUTLASS 推导链 (mma_atom.hpp) :
2275 //     make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMNK
2276 //     → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2277 //     → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2278 //     做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2279 //         K 被分为 BK/kMmaPK 个 perm-K slice
2280 //         随后按 AtomShape_MNK<2> (MMA_K_atom) 做 zipped_divide
2281 //         每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2282 //     → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2283 // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2284 int num_k_steps = size<2>(tCrA);
2285
2286 #pragma unroll
2287 for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2288     int k_step_next = (k_step + 1) % num_k_steps;
2289
2290     if (k_step == num_k_steps - 1) {
2291         // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2292         cp_async_wait<kStage - 2>();
2293         __syncthreads();
2294         ismem_read = (ismem_read + 1) % kStage;
2295     }
2296
2297     // S→R: 加载下一组 A/B fragment 到寄存器。
2298     //
2299     // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2300     //     cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2301     // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**:
2302     //     当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2303     //     让 S→R 延迟与 MMA 计算重叠。
2304     // 如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。
2305     cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2306               tCrA_view(_, _, k_step_next));
2307     cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2308               tCrB_view(_, _, k_step_next));
2309
2310     if (k_step == 0) {
2311         // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2312         if (itile_to_read < num_k_tiles) {
2313             cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2314                       tAsA_copy(_, _, _, ismem_write));
2315             cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2316                       tBsB_copy(_, _, _, ismem_write));
2317             ++itile_to_read;
2318             ismem_write = (ismem_write + 1) % kStage;
2319         }
2320         cp_async_fence();
2321     }
2322
2323     // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2324     // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2325     cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2326 }
2327 }
2328
2329 // Epilogue: D 寄存器 → shared memory → global memory。
2330 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。

```



```

2331 // 这里是“复用 storage”，不是把 A Tensor 转换成 C Tensor:
2332 //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2)，而 sA(.,.,ismem_read)
2333 //   只选出其中一个 A-stage 的 storage 起点；sC 随后以完全独立的 SmemLayoutC
2334 //   解释这同一段地址。当前 wrapper 的固定配置下:
2335 //       一个 A stage: 128 x 32 = 4096 个 half;
2336 //       C scratchpad: 32 x 32 x 4 = 4096 个 half。
2337 //   两者容量刚好相等，但 logical shape 和 layout 不是同一个: A 的 layout atom
2338 //   是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2339 //   C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2340 //   检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2341 //
2342 // mainloop 已结束，不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2343 // 从这一行开始，对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2344 // 用它写 C, S2G 用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2345 auto sC = make_tensor(sA(.,., ismem_read).data(), SmemLayoutC{});
2346
2347 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2348 // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2349 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2350 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2351 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2352 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位，但该 layout 是为 MMA
2353 // accumulator 服务的，并不直接按 R2S copy atom 的 source-value 顺序表达。
2354 //
2355 // retile_S 中的 S 指“这个 TiledCopy 的 source side”，不是 shared memory。
2356 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2357 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2358 // 或写入寄存器/共享内存，不进行 dtype conversion, 也不交换 lane 间数据；真正的
2359 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2360 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2361 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2362 // 角色，而不是另一次 memory transfer。
2363 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2364 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2365 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2366 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2367 //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2368 //       → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2369 //       → make_tiled_copy_C 的 tiler = (32, 32)
2370 //       → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2371 //       → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2372 //       → 结果 shape 的 M/N remainder = _1, _1
2373 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2374 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2375 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2376 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2377
2378 // S2G TiledCopy: shared memory → global memory (128-bit store)
2379 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2380 S2GCopyC s2g_tiled_copy_c;
2381 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2382 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生，因而保留相同的 pipe=4; 前者是
2383 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2384 // gD=(BM,BN)=(128,256)，相对于 S2G copy tiler 仍有非平凡的 M/N repetition,
2385 // 所以它保留 CPY_M, CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2386 // 的逻辑 mode, 不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2387 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2388 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2389
2390 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2391 //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2392 //       -> Tensor(CPY, (CPY_M, CPY_N))
2393 // 它只重组 Tensor 的 layout, 不分配内存、不移动 data(), 也不改变元素访问的物理地址。

```

```

2394 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系，只是现在可以用一个坐标
2395 // i+j 访问这个组合 mode；因此这里的 group_modes 更准确地说“合并 mode”，
2396 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2397 // 从 type/shape 结构看，结果确实是 Tensor(CPY, (CPY_M, CPY_N))；但 grouped
2398 // mode 的 cardinality 是两个子 mode 的乘积，因此 size<1>(…) 可作为一个标量
2399 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2400 // 的线性 logical coordinate，不是在 C++ 中对二维 tuple 做加法，更不是 raw
2401 // pointer offset；最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2402 //
2403 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group？
2404 //   - tCrC_r2s：每个线程持有的寄存器 C fragment，原始形状为 (CPY, CPY_M, CPY_N)；
2405 //   - tCgC_s2g：该线程对应的 global-memory C 输出位置，形状与源 Tensor 对齐；
2406 //   - 两者使用相同的 [1,3] 合并后，源/目的 Tensor 仍保持相同的坐标空间，
2407 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2408 //
2409 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode，因为它们还需要显式保留
2410 // shared-memory C scratchpad 的 pipeline 维度；step=size<3>(tCsC_r2s) 表示
2411 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2412 // 按 step 前进，内层 j 选择当前 pipeline slot：寄存器先写入 sC(_,0,0,j)，
2413 // 再从同一个 slot 写回 global memory。
2414 // 当前 kSmemLayoutCBatch=4，因此 step=4。代码未对 i+j 做尾部 predication，
2415 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除；不应将
2416 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2417 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2418 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2419
2420 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数（由 kSmemLayoutCBatch=4 指定）
2421 // 双层循环的语义（注意 step 与 CPY_N 无关）：
2422 //   group_modes<1,3> 之后，tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2423 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N，即该线程需处理的 C fragment 总数。
2424 //   外循环以 step=4 为步长，内循环 j=0..3 每次处理 1 个 fragment，将其写入
2425 //   第 j 号 C scratchpad pipe slot，再读出写回 global。
2426 //
2427 //   这里 step=4 是 scratchpad pipeline 深度，不是 CPY_N。CPY_N 的值取决于
2428 //   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上，
2429 //   通常 ≠ 4。循环之所以成立，是因为 grouped mode 用线性坐标 i+j 遍历整个
2430 //   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向，也不要求 CPY_N == step。
2431 //   唯一前提是 CPY_M * CPY_N 能被 step 整除（当前静态 layout 编译期保证）。
2432 //
2433 //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2434 //   各自推导；因为二者都从同一个 sC (SmemLayoutC) 出发，pipe mode 保持一致，
2435 //   写入和读取命中同一段 shared memory。
2436 #pragma unroll
2437 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2438     // 每轮处理 step 个 fragment，内层 j 选 pipe slot。
2439 #pragma unroll
2440     for (int j = 0; j < step; ++j) {
2441         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2442         // 的 register Tensor；普通 cute::copy 将 accumulator fragment materialize
2443         // 成输出元素类型 T 的临时 payload。
2444         //
2445         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2446         // dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2447         // 因此当 accumulator 与 T 类型不同时，它自动完成 dtype 转换；当二者相同
2448         // （如本 kernel 的 f16 accumulator + T=half），cast 退化为 no-op。
2449         //
2450         // 即使类型一致，t 还有一个不可省略的作用：layout 归一化。
2451         // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2452         // 叠加的 composed layout，而 make_tensor_like<T> + cute::copy 把它
2453         // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2454         // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2455         // 拆分 source 元素，简单 owning layout 比复杂的 composed layout 更容易
2456         // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and

```

```

2457 // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2458 //
2459 // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2460 // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2461 // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2462 // 由随后的 R2S -> shared memory -> S2G 路径完成。若特定 accumulator/output
2463 // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2464 // 保持同源实现的通用 staging 写法。
2465 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2466 cute::copy(tCrC_r2sx(_, i + j), t);
2467 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2468 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2469 //
2470 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2471 //   r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>, T>
2472 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2473 //   → src(t) 与 dst(tCsC_r2s(_, 0, 0, j)) 都是 rank-1 → 直接 copy_atom.call
2474 //   → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2475 //   走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2476 //   → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2477 //
2478 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2479 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2480 //   - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2481 //   → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2482 //   - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2483 //   → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2484 //   计算了当前线程每个 val 在 smem 中的物理 offset
2485 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2486 // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2487 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2488 }
2489 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2490 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2491 __syncthreads();
2492
2493 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2494 #pragma unroll
2495 for (int j = 0; j < step; ++j) {
2496 // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2497 // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2498 // (如 st.global.v4.f32 或 st.global.b128)。
2499 //
2500 // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2501 //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>, T>
2502 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2503 //   → src=tCsC_s2g(_, 0, 0, j) 与 dst=tCgC_s2gx(_, i+j) 都是 rank-1
2504 //   → 直接 copy_atom.call(src, dst)
2505 //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2506 //   → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2507 //   → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2508 //
2509 // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2510 //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2511 //   用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2512 //   在 sC (shared memory) 中的物理 offset。
2513 //   S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2514 //   = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2515 //   因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2516 //   M/N remainder。
2517 //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2518 //   → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2519 //   因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N

```

```

2520 // repetition 后得到 (CPY, (CPY_M,CPY_N)),再用线性坐标 i+j 切片。
2521 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2522 // 因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2523 cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2524 }
2525 // 所有线程都完成当前 pipe slots 的 S2G 读取后,下一轮 R2S 才能覆盖这 4 个
2526 // scratchpad slots,避免一部分线程仍在读取旧数据时被其他线程提前重写。
2527 __syncthreads();
2528 }
2529
2530 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照:手写版必须显式处理 MMA fragment
2531 // 的物理分布,使用 RC0/RC1 暂存,再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2532 // 8 个 half,并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2533 // CuTe 版没有把这些 lane 映射写在 kernel 源码中:retile_S 表达 accumulator
2534 // fragment 与 R2S copy 的对应,R2S/sC/S2G 通过 shared-memory scratchpad 完成
2535 // CTA 范围的数据重组,S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2536 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2537 // 区别是手写版直接编排 shuffle/store,CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2538 }
2539
2540 // =====
2541 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2542 // =====
2543 // CuTe 的核心设计理念:“类型即配置”(Type-level configuration)
2544 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2545 // kernel 接收这些类型的实例(通常为 struct),在编译期完成全部映射推导。
2546 //
2547 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法,形成 1:1 对照。
2548 //
2549 // 面试常问:「CuTe 为什么比手写 PTX 简洁?」
2550 // 答:手写需要:
2551 // 1) 手动计算每线程的 smem 地址偏移
2552 // 2) 手动计算 ldmatrix 的 lane 映射
2553 // 3) 手动插入 swizzle XOR 到每个地址计算点
2554 // 4) 手动做 epilogue 的 warp shuffle 编排
2555 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2556 // 在编译期自动完成上述全部推导,生成与手写等价的 PTX 指令序列。
2557 // =====
2558 template <typename T, const int Stages = 2>
2559 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2560     using namespace cute;
2561
2562     // — Tile 尺寸(kernel 模板参数 BM/BN/BK/kStage 的值) —
2563     //
2564     // kernel 用法对照:
2565     // BM=128: local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2566     //          local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2567     // BN=256: local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2568     //          local_tile(D, ...) → gD 的列方向
2569     // BK=32: num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2570     // kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2571     // kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2572     auto BM = Int<128>{};
2573     auto BN = Int<256>{};
2574     auto BK = Int<32>{};
2575     auto KStage = Int<Stages>{};
2576     auto kSmemLayoutCBatch = Int<4>{};
2577
2578     // — SmemLayoutA / SmemLayoutB —
2579     // kernel 用法:
2580     // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2581     // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2582     // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。

```



```

2583 //
2584 // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2585 //   - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2586 //   - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2587 //   - B=3: 共有 2^3=8 行。
2588 // 于是一个 8x32 的逻辑 tile 正好对应 8 行 x 128B, 覆盖全部 32 个 shared-memory bank。
2589 // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2590 // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2591 // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2592 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2593 // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2594 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2595 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2596 // 使结果 shape 匹配 BMxBKxkStage; 默认按目标 shape 的 mode order 重复各维。
2597 using SmemLayoutAtom = decltype(composition(
2598     Swizzle<3, 3, 3>{},
2599     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2600         make_stride(Int<BK>{}, Int<1>{}))));
2601 using SmemLayoutA = decltype(tile_to_shape(
2602     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2603 using SmemLayoutB = decltype(tile_to_shape(
2604     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2605
2606 // — MMA (TiledMMA) —
2607 // kernel 用法:
2608 // TiledMMA tiled_mma;
2609 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2610 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2611 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2612 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2613 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2614 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2615 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2616 //
2617 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2618 //   → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2619 //   → TiledMMA: 128 threads = 4 warps x (2x2 EU slices), 逻辑 MMA tile = 32x32x16
2620 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2621 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2622 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2623 using mma_traits = MMA_Traits<mma_op>;
2624 using mma_atom = MMA_Atom<mma_traits>;
2625 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2626
2627 static constexpr int kMmaEURepeatM = 2;
2628 static constexpr int kMmaEURepeatN = 2;
2629 static constexpr int kMmaEURepeatK = 1;
2630 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2631 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2632 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2633 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2634
2635 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2636     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2637 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2638 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2639
2640 // — G2SCopyA / G2SCopyB —
2641 // kernel 用法:
2642 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2643 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,istage), tAsA_copy(_,_,istage));
2644 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,istage), tBsB_copy(_,_,istage));
2645 // G→S: 128-bit cp.async. ThrLayout{32,4} x ValLayout{1,8}:

```

```

2646 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits。
2647 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2648 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2649 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2650 using G2SCopyA = decltype(make_tiled_copy(
2651     g2s_copy_atom{,
2652         make_layout(make_shape(Int<32>{}, Int<4>{}),
2653             make_stride(Int<4>{}, Int<1>{})),
2654         make_layout(make_shape(Int<1>{}, Int<8>{}))));
2655 using G2SCopyB = G2SCopyA;
2656
2657 // — S2RCopyAtomA / S2RCopyAtomB —
2658 // kernel 用法:
2659 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{, tiled_mma);
2660 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{, tiled_mma);
2661 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2662 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2663 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2664 // 与 MMA fragment 对齐的 shared→register 映射, 无需手动指定 ThrLayout/ValLayout。
2665 using s2r_copy_op = SM75_U32x4_LDSM_N;
2666 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2667 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2668 using S2RCopyAtomA = s2r_copy_atom;
2669 using S2RCopyAtomB = s2r_copy_atom;
2670
2671 // — SmemLayoutC —
2672 // kernel 用法:
2673 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2674 // 复用当前 A stage 的空间作为 C scratchpad
2675 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2676 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2677 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2678 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2679 // 的 kStage K-tile pipeline。
2680 using SmemLayoutAtomC = decltype(composition(
2681     Swizzle<3, 3, 3>{,
2682         make_layout(make_shape(Int<kMmaPM>{, Int<kMmaPN>{,
2683             make_stride(Int<kMmaPN>{, Int<1>{}})));
2684 using SmemLayoutC = decltype(tile_to_shape(
2685     SmemLayoutAtomC{,
2686         make_shape(Int<kMmaPM>{, Int<kMmaPN>{, Int<kSmemLayoutCBatch>{}}));
2687
2688 // — R2SCopyAtomC —
2689 // kernel 用法:
2690 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{, tiled_mma);
2691 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2692 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2693 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2694 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2695 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2696
2697 // — S2GCopyC —
2698 // kernel 用法:
2699 // S2GCopyC s2g_tiled_copy_c;
2700 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2701 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared→global wide store。
2702 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2703 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2704 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2705 using S2GCopyC = decltype(make_tiled_copy(
2706     S2GCopyAtomC{,
2707         make_layout(make_shape(Int<32>{, Int<4>{,
2708             make_stride(Int<4>{, Int<1>{}})),

```

```

2709     make_layout(make_shape(Int<1>{}, Int<8>{})));
2710
2711 // — Grid/Block 配置 —
2712 // kernel 用法:
2713 // int idx = threadIdx.x;          // 0..127
2714 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2715 // int iy = blockIdx.y;           // M 方向 CTA 坐标
2716 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2717 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2718 // M % BM == 0, N % BN == 0, K % BK == 0。
2719 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2720 constexpr int kBlockSwizzle = 0;
2721 constexpr int kSwizzleStride = 2048;
2722 int BX = (N + BN - 1) / BN;
2723 int BY = (M + BM - 1) / BM;
2724 int BZ = kBlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2725 BX = kBlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2726 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2727 dim3 grid(BX, BY, BZ);
2728
2729 // — Dynamic Shared Memory 大小 —
2730 // kernel 用法:
2731 // extern __shared__ T shm_data[];
2732 // T *Ashm = shm_data;
2733 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2734 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2735 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2736 // 合计 A+B = 24576 个 T; C epilogue 复用一个 A stage (128*32 = 4096 个 T)。
2737 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2738 // 能放进一个 A stage (128*32 = 4096)。
2739 static constexpr int shm_size_AB =
2740     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2741 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2742 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2743     "C shared memory must fit within one A pipe");
2744 static constexpr int kShmSize =
2745     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2746
2747 cudaFuncSetAttribute(
2748     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2749         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2750         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2751         S2GCopyAtomC, S2GCopyC, kBlockSwizzle>,
2752     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2753
2754 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2755     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2756     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2757     kBlockSwizzle>
2758     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2759 }
2760
2761 #endif /* NOTES_V2_ENABLE_CUTE */
2762
2763 // =====
2764 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2765 // =====
2766 // 面试要点 (WGMMMA vs MMA 对比):
2767 // - MMA: warp 级 (32 threads), 同步执行
2768 // - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
2769 // - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2770 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2771 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步

```



```

2772 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2773
2774 // ---- WGMMA 辅助函数 ----
2775 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2776 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2777 //   (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行)。
2778 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2779 //   N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
2780 //   都是 "wait until only N or fewer groups are pending"。
2781 #define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2782 #define WGMMA_COMMIT_GROUP() \
2783     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2784 #define WGMMA_WAIT_GROUP(n) \
2785     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2786
2787 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
2788 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2789 //   encoded = (x & 0x3FFFF) >> 4
2790 // 其中 0x3FFFF = 218 - 1 = 262143, 只保留低 18 bit。
2791 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (24), 低 4 bit 恒为 0,
2792 // 可省略以扩大可表示范围。
2793 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2794 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2795
2796 // make_smem_desc: 构造 WGMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2797 // 硬件 WGMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2798 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2799 //
2800 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2801 // -----
2802 // | 位域          | 范围      | 大小 | 含义
2803 // |-----|-----|-----|-----|
2804 // | start_address  | [0, 14)   | 14   | smem 基址编码
2805 // | (unused)       | [14, 16)  | 2    | -
2806 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2807 // | (unused)       | [30, 32)  | 2    | -
2808 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2809 // | (unused)       | [46, 49)  | 3    | -
2810 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2811 // | (unused)       | [52, 62)  | 10   | -
2812 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2813 // -----
2814 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2815 //
2816 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2817 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2818 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2819 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2820 // - 这是 stride_byte_offset=1024 的来源
2821 //
2822 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1),
2823 //   此处填 16 仅为占位, 编码后为 1。
2824 //
2825 // - base_offset=0 (bits [49,52) 未显式置位), 表示 smem ptr 已 1024B 对齐。
2826 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2827 //
2828 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2829 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2830 __device__ inline uint64_t make_smem_desc(half *ptr) {
2831     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2832     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2833     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2834

```

```

2835 // 从零开始: 所有位域初始化为 0。
2836 uint64_t desc = 0x0000000000000000;
2837
2838 // — bits [0, 14): start_address —
2839 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2840 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2841 // 可寻址范围:  $2^{(14+4)} = 2^{18} = 256K$  个 half = 512 KB 共享内存。
2842 desc |= SMEM_DESC_ENCODE(addr);
2843
2844 // — bits [16, 30): leading_byte_offset —
2845 // 原始值 16 (字节), 编码后为  $(16 \& 0x3FFFF) \gg 4 = 1$ 。
2846 // << 16 将编码值放到 bits [16, 30)。
2847 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2848 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2849 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2850 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2851 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2852 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2853
2854 // — bits [32, 46): stride_byte_offset —
2855 // 原始值 1024 (字节), 编码后为  $(1024 \& 0x3FFFF) \gg 4 = 64$ 。
2856 // << 32 将编码值放到 bits [32, 46)。
2857 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
2858 // 1024 的来源 (K-Major + 128B swizzle + half) :
2859 // 一个 128B swizzle atom 覆盖  $64(K) \times 8(M)$  个 half。
2860 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越  $8 \times 64 \times 2 = 1024$  bytes。
2861 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2862 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2863
2864 // — bits [62, 64): layout_type (swizzle mode) —
2865 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2866 // 1 = 128B swizzle mode。
2867 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2868 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2869 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2870 desc |= 1llu << 62;
2871
2872 return desc;
2873 }
2874
2875 // ---- WGMMMA PTX 指令宏 ----
2876 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2877 // m64n128k16: M=64, N=128, K=16。一次 WGMMMA 计算  $D[64 \times 128] += A[64 \times 16] \times B[16 \times 128]$ 。
2878 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2879 // 每线程 32 个 uint32 输出:  $D=64 \times 128=8192$  half, warpgroup 128 线程分担,
2880 // 每线程 64 half = 32 uint32。
2881 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2882 // Scaled: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2883 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2884 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2885 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2886 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, Scaled, ScaleA, ScaleB, TransA, \
2887                                     TransB) \
2888 { \
2889     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2890     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2891     asm volatile( \
2892         "{\n" \
2893         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2894         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2895         " %8, %9, %10, %11, %12, %13, %14, %15, " \
2896         " %16, %17, %18, %19, %20, %21, %22, %23, " \
2897         " %24, %25, %26, %27, %28, %29, %30, %31}, " \

```

```

2898     " %32," \
2899     " %33," \
2900     " %34, %35, %36, %37, %38;\n" \
2901     "}\n" \
2902     : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2903     "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2904     "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2905     "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2906     "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2907     "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2908     "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2909     "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2910     : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
2911     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2912     "n"(int32_t(TransB))); \
2913 }
2914
2915 // ---- TMA Shared Memory Layout ----
2916 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2917 //
2918 // 每个 stage 包含两块 smem:
2919 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2920 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
2921 //
2922 // ★ 关键区别 — WGMMMA 的 B 物理存储 vs 逻辑视图:
2923 //
2924 //   上层数据: 源矩阵 B^T [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
2925 //   物理存储: TMA 将 B^T 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
2926 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2927 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2928 //   TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2929 //   逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2930 //   通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2931 //   实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2932 //   swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2933 //
2934 //   stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2935 //       128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2936 //       stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2937 //                           = 8 rows × (BK=64) halves × 2 bytes = 1024。
2938 //       这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2939 //
2940 //   与 MMA(TN) 的对比:
2941 //       MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2942 //       WGMMMA:   smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2943 //       简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
2944 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
2945     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
2946     // B tile 标记:
2947     //   - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
2948     //   - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
2949     //       以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
2950     //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
2951     //       但写 BN×BK 更能体现物理存储形态
2952     alignas(128) half B[BN * BK * QSIZE];
2953 };
2954
2955 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
2956 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
2957 //
2958 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
2959 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
2960 // warpgroup, 让数据搬运和矩阵乘完全异步执行:

```

```

2961 //
2962 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
2963 //   将 A/B tile 从 HBM 搬到 shared memory。
2964 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
2965 //
2966 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
2967 //   - full[qidx]: Producer 发信号表示 stage qidx 的数据已就绪, 可被使用
2968 //   - empty[qidx]: Consumer 发信号表示 stage qidx 的使用完毕, 可以被覆盖
2969 //
2970 // 本节重点理解:
2971 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
2972 //       即可搬运整个 2D tile, 无需所有线程参与。
2973 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
2974 //   3) 多 stage pipeline 使 Consumer 计算 stage qidx 的同时,
2975 //       Producer 可以搬运 stage (qidx+1), 隐藏 HBM→SMEM 延迟。
2976 //
2977 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
2978 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
2979 //   K Tile:           BK=64, 每个 K tile 包含 BK/kWmmaK=4 个 WGMMMA atom
2980 //   M Tile:           BM=128, 每个 M tile 包含 BM/kWmmaM=2 个 WGMMMA atom
2981 //   N Tile:           BN=128, 单个 kWmmaN=128 即可覆盖, 无需在 N 方向分块
2982 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
2983 //   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
2984 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
2985 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)
2986 //
2987 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
2988 // 的延迟被计算完全掩盖。
2989 //
2990 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
2991 //   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
2992 // Block: (256, 1, 1), 2 warpgroups
2993 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
2994 template <const int kWmmaM = 64,           // WGMMMA atom M dim (m64n128k16)
2995           const int kWmmaN = 128,         // WGMMMA atom N dim
2996           const int kWmmaK = 16,          // WGMMMA atom K dim
2997           const int BM = 128,             // block tile M, 2 × kWmmaM
2998           const int BN = 128,             // block tile N, 1 × kWmmaN
2999           const int BK = 64,              // block tile K, 4 × kWmmaK
3000           const int kNumThreads = 256,    // 2 warpgroups × 128 threads
3001           const int kStages = 3,          // TMA pipeline depth (full/empty barriers)
3002           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
3003 __global__ void __launch_bounds__(kNumThreads)
3004   hgemm_wgmma_stages_tn(
3005     int M, int N, int K, half *C,
3006     const CUtensorMap *__restrict__ tensorMapA,
3007     const CUtensorMap *__restrict__ tensorMapB) {
3008   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3009   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
3010   // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
3011   // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
3012   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
3013   // 不展开宿主侧 create_tensor_map 细节。
3014
3015   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
3016   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
3017   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
3018   const int by = blockIdx.y;
3019   // num_consumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
3020   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
3021   constexpr int num_consumers = (kNumThreads / 128) - 1; // 1 consumer WG
3022   // B_WG_M = BM / num_consumers: 每个 consumer warpgroup 负责的 M 行数
3023   // 当只有一个 consumer 时, 它负责全部 BM=128 行

```

```

3024 constexpr int B_WG_M = BM / num_consumers; // 128
3025
3026 // 边界检查: 确保当前 block 不超出 M/N 范围
3027 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3028     return;
3029
3030 // ---- Shared Memory 分配 ----
3031 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
3032 // __align__(128) 满足 TMA 和 WGMMMA 的 16B 对齐 + 128B swizzle 对齐要求
3033 extern __shared__ __align__(128) uint8_t smem_AB[];
3034 WgmmaSMem<BM, BN, BK, kStages> &s =
3035     *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages>*>(smem_AB);
3036 half *s_a = s.A;
3037 half *s_b = s.B;
3038
3039 // ---- cuda::barrier 初始化 ----
3040 //
3041 // ★ Barrier 机制速查 (arrive / wait 核心语义):
3042 //
3043 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
3044 //
3045 //   init(bar, N): 设置 arrive_count = N。
3046 //       含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
3047 //
3048 //   arrive(): 线程声明"我已到达此 barrier 点"。
3049 //       - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3050 //       - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3051 //
3052 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3053 //       - 即: 等待 phase 翻转。
3054 //       - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3055 //                           (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3056 //
3057 //   典型用法: b.wait(b.arrive())
3058 //       - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3059 //       - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3060 //       - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3061 //       - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3062 //
3063 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129):
3064 //
3065 //   Producer (1 thread)           Consumer (128 threads)
3066 //   ────────────                 ────────────
3067 //   C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3068 //   ┌ for each k_tile: ┐           ┌ for each k_tile: ┐
3069 //   │ P1: arrive+wait(empty[q]) │   │ C1: arrive+wait(full[q]) │
3070 //   │   ↓ 等 stage q 被消费完   │   │   ↓ 等 TMA 数据就绪   │
3071 //   │ P2: TMA(A[q]) + TMA(B[q]) │   │ C2: WGMMMA 计算   │
3072 //   │   ↓ 异步拷贝             │   │ C3: wait WGMMMA 完成   │
3073 //   │ P3: arrive_tx(full[q])    │   │ C4: arrive(empty[q]) │
3074 //   │   ↑ 通知: stage q 数据就绪 │   │   ↑ 通知: stage q 已消费完 │
3075 //   └───────────────────┘         └───────────────────┘
3076 //
3077 // 关键不变式 (invariant):
3078 //   - Producer 不会覆盖 Consumer 正在读的 stage
3079 //   - Consumer 不会读 Producer 还没写完的 stage
3080 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
3081 //
3082 // 每个 stage 有两个 barrier:
3083 //   full[qidx]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3084 //   empty[qidx]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3085 //
3086 // arrive_count = 128 (consumer) + 1 (producer) = 129:

```



```

3087 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3088 #pragma nv_diag_suppress static_var_with_dynamic_init
3089 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3090 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3091 #pragma nv_diag_default static_var_with_dynamic_init
3092
3093 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3094 const int num_blocks_k = K / BK;
3095 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
3096 const int tid = threadIdx.x % 128; // 0~127 within warpgroup
3097
3098 // 初始化 barriers (仅 thread 0 执行)
3099 if (threadIdx.x == 0) {
3100     for (int i = 0; i < kStages; ++i) {
3101         // arrive_count = 129: 128 consumer + 1 producer
3102         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
3103         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3104         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3105         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
3106         init(&empty[i], num_consumers * 128 + 1); // same
3107     }
3108     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3109     // 对 async proxy (TMA/WGMMMA) 可见。
3110     cuda::device::experimental::fence_proxy_async_shared_cta();
3111 }
3112 __syncthreads();
3113
3114 // =====
3115 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3116 // =====
3117 //
3118 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
3119 // - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
3120 // - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
3121 //
3122 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
3123 // Producer: for (k_iter = 0 .. num_blocks_k) { P1→P2→P3 } ← 自己的循环
3124 // Consumer: for (k_iter = 0 .. num_blocks_k) { C1→C2→C3→C4 } ← 自己的循环
3125 //
3126 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
3127 // - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
3128 // - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
3129 //
3130 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
3131 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
3132 // =====
3133 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3134 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
3135 // 只有 tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
3136 // =====
3137 if (wg_idx == 0) {
3138     if (tid == 0) {
3139         // qidx: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3140         int qidx = 0;
3141         for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
3142             if (qidx == kStages)
3143                 qidx = 0;
3144
3145             // Step P1: 等待 stage qidx 变为"空" (可被覆盖写入)
3146             //
3147             // 模式 empty[qidx].wait(empty[qidx].arrive()):
3148             // - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
3149             // 获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了

```



```

3150 // 128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
3151 // → phase 立即翻转。
3152 // - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3153 // 由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
3154 // wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3155 //
3156 // 语义: Producer 说"我准备好写 stage qidx 了，Consumer 用完没有？"
3157 // 如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3158 // 如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3159 empty[qidx].wait(empty[qidx].arrive());
3160
3161 // Step P2: 提交 TMA 2D 拷贝指令
3162 // cp_async_bulk_tensor_2d_global_to_shared:
3163 // 参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3164 // 参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3165 // 参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3166 // 参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3167 //
3168 // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile，
3169 // 无需线程逐元素搬运，零寄存器开销。
3170 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3171 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3172     &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
3173     full[qidx]);
3174
3175 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3176 cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
3177     &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
3178     full[qidx]);
3179
3180 // Step P3: 通知 Consumer: stage qidx 的 TMA 数据已就绪
3181 //
3182 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3183 // - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3184 // - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3185 // - Phase 翻转条件:
3186 // (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3187 // (b) 所有声明的 async 字节已写入 smem
3188 // 两个条件都满足后 phase 才翻转，Consumer 的 wait() 才返回。
3189 // - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3190 [[maybe_unused]] auto token = cuda::device::barrier_arrive_tx(
3191     full[qidx], 1, (BK * BN + BK * BM) * sizeof(half));
3192 }
3193 }
3194 }
3195 // =====
3196 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3197 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3198 // 所有 128 个线程 (4 warps) 全部参与。
3199 // =====
3200 else {
3201 // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3202 //
3203 // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3204 // 这是 Pipeline 的"预热"步骤——没有它，Producer 的 empty[qidx].wait()
3205 // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3206 //
3207 // 注意: 此时每个 empty[i] 只有 128 次 arrive，未达 129，phase 不翻转。
3208 // Producer 后续的 empty[qidx].arrive() 作为第 129 次，触发 phase 翻转。
3209 for (int i = 0; i < kStages; ++i) {
3210     [[maybe_unused]] auto token = empty[i].arrive();
3211 }
3212

```

```

3213 // 累加器寄存器声明
3214 // d[B_WG_M / kWgmmaM][kWgmmaN / 16][4]:
3215 // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3216 // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3217 // - d[*][g][*]: N 方向第 g 组 16 列
3218 // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3219 // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3220 // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3221 uint32_t d[B_WG_M / kWgmmaM][kWgmmaN / 16][4] = {};
3222
3223 int qidx = 0;
3224 // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3225 for (int block_k_iter = 0; block_k_iter < num_blocks_k; ++block_k_iter, ++qidx) {
3226     if (qidx == kStages)
3227         qidx = 0;
3228
3229     // Step C1: 等待 TMA 数据就绪 (full 信号)
3230     //
3231     // 模式 full[qidx].wait(full[qidx].arrive()):
3232     // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3233     // - 当前 phase 累积: 128/129, phase 尚未翻转。
3234     // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3235     //   贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3236     //
3237     // 语义: Consumer 说"我准备好读 stage qidx 了, 数据到了没有?"
3238     full[qidx].wait(full[qidx].arrive());
3239
3240     // Step C2: 发射 WGMMMA 指令序列
3241     //
3242     // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3243     // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3244     //
3245     // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3246     // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3247     // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3248     // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3249     //
3250     // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3251     // D[64x128] += A[64x16] × B[16x128]
3252     // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3253     // ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都 ScaleD=1。
3254     WGMMMA_FENCE();
3255
3256     // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMMA atom
3257     // 每个 atom 处理 64 行 M 方向数据。
3258 #pragma unroll
3259     for (int m_it = 0; m_it < B_WG_M / kWgmmaM; ++m_it) {
3260         // wgmma_sA 指向当前 stage 中 A tile 的第 m_it 个 64*BK 子块
3261         // s_a 布局: [kStages][BM][BK] -> qidx * BK*BM + BK * (m_it*64)
3262         half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * kWgmmaM;
3263
3264         // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMMA (累加)
3265         // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3266 #pragma unroll
3267         for (int k_it = 0; k_it < BK / kWgmmaK; ++k_it) {
3268             // 第 k_it 次 WGMMMA:
3269             // A: wgmma_sA + k_it * kWgmmaK (A 的 K 维起始位置)
3270             // B: s_b + qidx * BK * BN + k_it * kWgmmaK (B 的 K 维起始位置)
3271             // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3272             // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3273             // ScaleD=1: 累加 (K 维迭代需要累积结果)
3274             WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * kWgmmaK,
3275                 s_b + qidx * BK * BN + k_it * kWgmmaK,

```

```

3276         1, // Scaled=1: accumulate (不清零)
3277         1, 1, 0, 0);
3278     }
3279 }
3280
3281 // Step C3: 提交并等待 WGMMMA 完成
3282 //
3283 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3284 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3285 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3286 //
3287 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3288 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3289 // * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3290 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
3291 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3292 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3293 WGMMMA_COMMIT_GROUP();
3294 WGMMMA_WAIT_GROUP(0);
3295
3296 // Step C4: 释放 stage qidx — 通知 Producer 可以覆盖写入
3297 //
3298 // 128 个 Consumer 线程在 empty[qidx] 上 arrive(), 为 **下一 phase** 积累到达次数。
3299 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3300 //
3301 // 语义: Consumer 说"stage qidx 我已经读完了, 你可以放心覆盖了。"
3302 [[maybe_unused]] auto token = empty[qidx].arrive();
3303 }
3304
3305 // =====
3306 // Epilogue: 将寄存器中的累加结果写回 global memory C
3307 // =====
3308 //
3309 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3310 //
3311 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3312 // 这些 half 分布在 d[2][8][4] 数组中:
3313 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3314 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3315 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3316 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3317 //
3318 // 其中:
3319 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
3320 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3321 //
3322 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3323 // +-----+-----+
3324 // | reg[0] | reg[2] | <- rows [row, row+8)
3325 // |(col,+2)|(col+8)|
3326 // +-----+-----+
3327 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3328 // |(col,+2)|(col+8)|
3329 // +-----+-----+
3330 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3331 //
3332 // 三维遍历:
3333 // m_it=0: rows 0~63, m_it=1: rows 64~127
3334 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3335 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3336 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3337
3338 const int lane = tid % 32;

```

```

3339     const int warp = tid / 32;
3340     // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3341     // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3342     // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3343     //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3344     //         因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3345     //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3346     const int row = warp * 16 + lane / 4;
3347     // block_C: 当前 C tile 在 global memory 中的起始地址
3348     // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3349     half *block_C = C + by * BM * N + bx * BN;
3350
3351     // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3352 #pragma unroll
3353     for (int m_it = 0; m_it < B_WG_M / kWgmmaM; ++m_it) {
3354         int yo = m_it * kWgmmaM; // M 方向行偏移 (0 或 64)
3355 #pragma unroll
3356         for (int g = 0; g < kWgmmaN / 16; ++g) {
3357             int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3358             // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3359             // 左上象限: (row, col)
3360             *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
3361                 d[m_it][g][0];
3362             // 左下象限: (row+8, col)
3363             *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
3364                 d[m_it][g][1];
3365             // 右上象限: (row, col+8)
3366             *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
3367                 d[m_it][g][2];
3368             // 右下象限: (row+8, col+8)
3369             *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
3370                 d[m_it][g][3];
3371         }
3372     }
3373 }
3374 }
3375
3376 #if (defined(NOTES_V2_ENABLE_WGMMMA))
3377 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3378 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
3379 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
3380 //   以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3381 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3382 //   对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3383 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3384 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3385 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3386 //
3387 // ★ gmem_prob_stride + 1 的含义:
3388 // 语法层面 — 数组名退化 + 指针算术:
3389 //   gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3390 //   (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3391 //   gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3392 // 语义层面 — 跳过隐式的最内维 stride:
3393 //   cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3394 //   stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3395 //   固定等于 sizeof(element_type), 不需要显式传入。
3396 //   对于 tensorRank=2 的 FP16 矩阵:
3397 //       dim 0 (内维, K): stride = sizeof(half) ← 隐式, API 不需要
3398 //       dim 1 (外维, M): stride = sizeof(half)*K ← 需要传给 API
3399 //   gmem_prob_stride 数组的布局是内维在前:
3400 //       [0] = sizeof(half) ← 内维, 不传
3401 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API

```

```

3402 // 所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3403 //
3404 // * smem_box_stride 为什么不需要 +1?
3405 // 与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3406 // 最内维 stride 也必须显式传入 (不隐式)。
3407 // smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3408 // 都是连续存放的, 维度间没有 padding/stride。
3409 // 对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
3410 // 即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3411 // 所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
3412 // 两个 stride 值, 不需要跳过任何一个。
3413 //
3414 // 参考: CUDA Programming Guide $TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
3415
3416 template <int BlockMajorSize, int BlockMinorSize>
3417 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3418                                             half *gmem_ptr,
3419                                             int blocks_height,
3420                                             int blocks_width) {
3421     void *gmem_address = (void *)gmem_ptr;
3422     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3423                                     (uint64_t)BlockMajorSize * blocks_height,
3424                                     1, 1, 1};
3425     uint64_t gmem_prob_stride[5] = {
3426         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3427     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3428                                    uint32_t(BlockMajorSize), 1, 1, 1};
3429     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3430     CUresult result = cuTensorMapEncodeTiled(
3431         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3432         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3433         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3434         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3435     if (result != CUDA_SUCCESS)
3436         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3437 }
3438
3439 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(
3440     half *src, int blocks_height, int blocks_width) {
3441     CUtensorMap *tma_map_d;
3442     cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
3443     CUtensorMap tma_map_host;
3444     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
3445     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
3446               cudaMemcpyHostToDevice);
3447     return tma_map_d;
3448 }
3449 #endif /* NOTES_V2_ENABLE_WGMMA */
3450
3451 // =====
3452 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
3453 // =====
3454 // 面试要点 (FlashAttention 算法):
3455 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3456 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
3457 // 2. FA 三板斧:
3458 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
3459 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3460 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3461 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3462 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3463 //    - 优点: 减少 warp 间通信和 shuffle
3464 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691):

```

```

3465 //      for each K,V tile:
3466 //          S_cur = Q @ K^T                                // 未缩放, 存入 R_S
3467 //          m_new = max(m_old, row_max(S_cur * scale))
3468 //          P_cur = exp(S_cur * scale - m_new)              // ← 写回 R_S 寄存器!
3469 //          l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3470 //          O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3471 //          O_final = O_new / l_final
3472 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3473 //     - R_S 经过 softmax 后, 存储的是 P = exp(S - m), 数据仍然是 half 精度
3474 //     - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3475 //       后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3476 //     - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3477 //       都天然同构”的通用结论
3478 //
3479 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3480 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3481 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3482 // Block: (128, 1, 1), kNumThreads=kWarpSize×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
3483 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3484
3485 // ---- 寄存器填充辅助函数 ----
3486 template <typename T, int M, const int N, const int K = 2>
3487 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3488     #pragma unroll
3489     for (int i = 0; i < M; ++i)
3490     #pragma unroll
3491         for (int j = 0; j < N; ++j)
3492     #pragma unroll
3493         for (int k = 0; k < K; ++k)
3494             R[i][j][k] = val;
3495 }
3496
3497 template <typename T, int M, const int N = 2>
3498 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3499     #pragma unroll
3500     for (int i = 0; i < M; ++i)
3501     #pragma unroll
3502         for (int j = 0; j < N; ++j)
3503             R[i][j] = val;
3504 }
3505
3506 // =====
3507 // FlashAttention-2 Split-Q Kernel (完整实现)
3508 // =====
3509 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3510 //
3511 // Tile 设计 (以 kHeadDim=64 为例) :
3512 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3513 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3514 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3515 //
3516 // 执行流程:
3517 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3518 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3519 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3520 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMM16816
3521 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3522 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3523 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3524 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3525 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3526 //   8) 最终 rescale: O_final = (1/l_final) * O_final
3527 //   9) Epilogue: warp shuffle + 128-bit collective store

```



```

3528
3529 template <
3530     const int kHeadDim,          // head dim: 32, 64, 128
3531     const int kMmaAtomM,        // 16 (MMA instruction M dimension)
3532     const int kMmaAtomN,        // 8 (MMA instruction N dimension)
3533     const int kMmaAtomK,        // 16 (MMA instruction K dimension)
3534     const int kMmaTileSeqLenQ,  // MMA tiles along Q's M dim, 4 → Br=16*4=64
3535     const int kMmaTileSeqLenK,  // MMA tiles along K's N dim, 1 → Bc basis=8
3536     const int kMmaTileSeqLenP,  // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3537     const int kMmaTileHeadDimV, // MMA tiles for P@V N dim (head dim direction)
3538     const int kValTileSeqLenQ,  // value tiles along Q's M, 1 → Br per warp=16
3539     const int kValTileSeqLenK,  // value tiles along K's N, 8 → Bc_warp=8*8=64
3540     const int kValTileSeqLenP,  // value tiles for P@V M dim, 1
3541     const int kValTileHeadDimV, // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3542     const int kStage,           // pipeline stages for K: 1 or 2
3543     const int kPad>            // padding for bank conflict avoidance
3544 __global__ void __launch_bounds__(kWarpSize *kMmaTileSeqLenQ *kMmaTileSeqLenK)
3545     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3546                                     int QKV_seqlen, int QKV_head) {
3547     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3548     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3549     constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3550     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3551     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3552     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3553     const float scale = 1.0f / sqrtf((float)kHeadDim);
3554
3555     // Block indexing
3556     const int QKV_batch_id = blockIdx.y / QKV_head;
3557     const int QKV_head_id = blockIdx.y % QKV_head;
3558     const int Q_tile_id = blockIdx.x;
3559     const int tid = threadIdx.x;
3560     const int warp_id = tid / kWarpSize;
3561     const int lane_id = tid % kWarpSize;
3562     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3563     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3564
3565     // Global memory base offsets for this (batch, head)
3566     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3567     const int Q_gmem_offset =
3568         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3569         kHeadDim;
3570     const int K_gmem_offset = Q_gmem_offset;
3571     const int V_gmem_offset = Q_gmem_offset;
3572     const int O_gmem_offset = Q_gmem_offset;
3573
3574     // Thread-to-smem mapping for cooperative load
3575     int load_smem_Q_Br = tid / (kNumThreads / Br);
3576     int load_smem_Q_d =
3577         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3578     int load_smem_K_Bc = tid / (kNumThreads / Bc);
3579     int load_smem_K_d =
3580         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3581     int load_smem_V_Bc = tid / (kNumThreads / Bc);
3582     int load_smem_V_d =
3583         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3584
3585     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3586     if (load_gmem_Q_Br >= QKV_seqlen)
3587         return;
3588
3589     // ---- Shared memory layout ----
3590     extern __shared__ half smem[];

```

```

3591 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3592 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3593 half *Q_tile_smem = smem;
3594 half *K_tile_smem = Q_tile_smem + Q_tile_size;
3595 half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3596 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3597 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3598
3599 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3600 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3601 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3602
3603 // ---- Online Softmax persistent state ----
3604 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3605 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3606 float lane_block_row_max_old[kValTileSeqLenQ][2];
3607 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3608 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3609 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3610
3611 // ---- Register allocation ----
3612 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3613 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3614 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3615 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3616 // 对单个 m16n8k16 tile 而言:
3617 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3618 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3619 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3620 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3621 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3622 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3623           [2]; // O accumulator (final output)
3624
3625 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3626 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3627
3628 // =====
3629 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3630 // =====
3631 {
3632     int load_gmem_Q_addr =
3633         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3634     uint32_t load_smem_Q_ptr =
3635         smem_Q_base_ptr +
3636         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3637 #pragma unroll
3638     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
3639         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
3640     }
3641     CP_ASYNC_COMMIT_GROUP();
3642 }
3643
3644 // =====
3645 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
3646 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
3647 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
3648 // =====
3649 if constexpr (kStage > 1) {
3650 #pragma unroll
3651     for (int stage = 0; stage < (kStage - 1); ++stage) {
3652         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
3653         int load_gmem_K_addr =

```

```

3654         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3655     uint32_t load_smem_K_ptr =
3656         smem_K_base_ptr +
3657         (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3658         load_smem_K_d) *
3659         sizeof(half);
3660 #pragma unroll
3661     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3662         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3663     }
3664     CP_ASYNC_COMMIT_GROUP();
3665 }
3666 CP_ASYNC_WAIT_GROUP(kStage - 2);
3667 __syncthreads();
3668 }
3669
3670 // =====
3671 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
3672 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
3673 // =====
3674 #pragma unroll 1
3675 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
3676     int smem_sel = tile_K_seqlen % kStage;
3677     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
3678
3679     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
3680     if constexpr (kStage > 1) {
3681         // 只有 kStage>1 才能真正做 K 的 pipeline:
3682         // smem_sel 负责“当前正在计算”的 K tile, smem_sel_next 负责“下一轮预取”的 K tile。
3683         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
3684         // Load current V tile (no pipeline for V — one stage is enough)
3685         {
3686             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
3687             int load_gmem_V_addr =
3688                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
3689             uint32_t load_smem_V_ptr =
3690                 smem_V_base_ptr +
3691                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
3692 #pragma unroll
3693             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3694                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
3695             }
3696             CP_ASYNC_COMMIT_GROUP();
3697         }
3698
3699         // Prefetch next K tile (pipelined)
3700         if ((tile_K_seqlen + 1) < Tc) {
3701             int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
3702             int load_gmem_K_addr =
3703                 K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
3704             uint32_t load_smem_K_ptr =
3705                 smem_K_base_ptr +
3706                 (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
3707                 load_smem_K_d) *
3708                 sizeof(half);
3709 #pragma unroll
3710             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
3711                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
3712             }
3713             CP_ASYNC_COMMIT_GROUP();
3714         }
3715     }
3716 }

```

```

3717 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
3718 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3719 #pragma unroll
3720 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
3721 // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
3722 #pragma unroll
3723 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3724 int warp_smem_Q_Br =
3725 warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
3726 int lane_smem_Q_Br =
3727 warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
3728 int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
3729 uint32_t lane_smem_Q_ptr =
3730 smem_Q_base_ptr +
3731 (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
3732 LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
3733 lane_smem_Q_ptr);
3734 }
3735
3736 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
3737 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
3738 #pragma unroll
3739 for (int j = 0; j < kValTileSeqLenK; ++j) {
3740 int warp_smem_K_Bc =
3741 warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
3742 int lane_smem_K_Bc =
3743 warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
3744 int lane_smem_K_d =
3745 tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
3746 uint32_t lane_smem_K_ptr =
3747 smem_K_base_ptr +
3748 (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
3749 lane_smem_K_d) *
3750 sizeof(half);
3751 LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
3752 }
3753
3754 // MMA: S[tile] += Q[tile] @ K^T[tile]
3755 #pragma unroll
3756 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3757 #pragma unroll
3758 for (int j = 0; j < kValTileSeqLenK; ++j) {
3759 HMMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
3760 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
3761 R_S[i][j][1]);
3762 }
3763 }
3764 } // end loop over d
3765 __syncthreads();
3766
3767 // =====
3768 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
3769 // =====
3770 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
3771 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
3772 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
3773 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
3774 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
3775 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
3776 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
3777 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
3778 // kValTileSeqLenK tiles.
3779

```

```

3780 float lane_row_max_new[kValTileSeqLenQ][2];
3781 float lane_row_sum_new[kValTileSeqLenQ][2];
3782 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
3783 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
3784
3785 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
3786 #pragma unroll
3787 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3788     #pragma unroll
3789     for (int j = 0; j < kValTileSeqLenK; ++j) {
3790         // Extract half values from R_S registers (C matrix fragment layout)
3791         float2 t_reg_S_0 =
3792             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
3793         float2 t_reg_S_1 =
3794             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
3795         // S = (Q@K^T) * scale
3796         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
3797         float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
3798         lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
3799         lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
3800     }
3801     // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
3802     // {0,4,8,...,28} hold valid data)
3803     lane_row_max_new[i][0] =
3804         warp_reduce_max<4, float>(lane_row_max_new[i][0]);
3805     lane_row_max_new[i][1] =
3806         warp_reduce_max<4, float>(lane_row_max_new[i][1]);
3807 }
3808
3809 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
3810 // 面试关键点: 这里将 P 写回 R_S 寄存器!
3811 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
3812 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
3813 #pragma unroll
3814 for (int i = 0; i < kValTileSeqLenQ; ++i) {
3815     // m_new = max(m_old, m_cur)
3816     float block_row_max_new_0 =
3817         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
3818     float block_row_max_new_1 =
3819         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
3820
3821     #pragma unroll
3822     for (int j = 0; j < kValTileSeqLenK; ++j) {
3823         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
3824         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
3825
3826         // P = exp(S * scale - m_new), 用 fma 保证精度
3827         t_reg_S_0.x =
3828             __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
3829         t_reg_S_0.y =
3830             __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
3831         t_reg_S_1.x =
3832             __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
3833         t_reg_S_1.y =
3834             __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
3835
3836         // Accumulate row sums
3837         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
3838         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
3839
3840         // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
3841         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
3842         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);

```

```

3843     }
3844
3845     // Warp-level reduce sum (kWarpWidth = 4, same as max)
3846     lane_row_sum_new[i][0] =
3847         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
3848     lane_row_sum_new[i][1] =
3849         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
3850 }
3851 __syncthreads();
3852
3853 // =====
3854 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
3855 // =====
3856 // Wait for V to be ready before computing P@V
3857 if constexpr (kStage > 1) {
3858     if ((tile_K_seqlen + 1) < Tc) {
3859         CP_ASYNC_WAIT_GROUP(1);
3860     } else {
3861         CP_ASYNC_WAIT_GROUP(0);
3862     }
3863 } else {
3864     CP_ASYNC_WAIT_GROUP(0);
3865 }
3866 __syncthreads();
3867
3868 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
3869
3870 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
3871 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
3872 #pragma unroll
3873 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
3874     // ldmatrix.x2.trans: load V[Bc,d] with transposition
3875     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
3876     // transposed ldmatrix
3877 #pragma unroll
3878 for (int j = 0; j < kValTileHeadDimV; ++j) {
3879     int warp_smem_V_d =
3880         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
3881     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
3882     int lane_smem_V_d = warp_smem_V_d;
3883     uint32_t lane_smem_V_ptr =
3884         smem_V_base_ptr +
3885         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
3886     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
3887 }
3888
3889 // P matrix layout in R_S[i][j][2]:
3890 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
3891 // | 64x64 | warp_KV 0 |
3892 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
3893 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
3894 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
3895 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
3896 // tile_V_Bc selects which 16-column slice of P to use:
3897 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
3898 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
3899 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
3900 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
3901 // 对应的 MMA A fragment 布局可以这样记:
3902 // - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
3903 // - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
3904 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
3905 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA

```



```

3906 // fragment 都无条件成立的通用结论。layout转换逻辑:
3907 // C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
3908 int w = tile_V_Bc * 2;
3909 #pragma unroll
3910 for (int i = 0; i < kValTileSeqLenP; ++i) {
3911 #pragma unroll
3912 for (int j = 0; j < kValTileHeadDimV; ++j) {
3913     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
3914             R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
3915             R_V[j][0], R_V[j][1], // B fragment = V
3916             R_0[i][j][0], R_0[i][j][1]); // C fragment output
3917     }
3918 }
3919 } // end for tile_V_Bc
3920 __syncthreads();
3921
3922 // =====
3923 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
3924 // =====
3925 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
3926 #pragma unroll
3927 for (int i = 0; i < kValTileSeqLenP; ++i) {
3928     float block_row_max_new_0 = lane_row_max_new[i][0];
3929     float block_row_max_new_1 = lane_row_max_new[i][1];
3930     float block_row_sum_new_0 = lane_row_sum_new[i][0];
3931     float block_row_sum_new_1 = lane_row_sum_new[i][1];
3932
3933     float block_row_max_old_0 = lane_block_row_max_old[i][0];
3934     float block_row_max_old_1 = lane_block_row_max_old[i][1];
3935
3936     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
3937     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
3938
3939     // Handle first iteration: m_old = -inf, need to use m_new directly
3940     block_row_max_old_0 =
3941         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
3942     block_row_max_old_1 =
3943         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
3944
3945     float rescale_o_factor_0 =
3946         __expf(block_row_max_old_0 - block_row_max_new_0);
3947     float rescale_o_factor_1 =
3948         __expf(block_row_max_old_1 - block_row_max_new_1);
3949
3950     // Rescale 0_old + Add P@V in one fused step
3951 #pragma unroll
3952 for (int j = 0; j < kValTileHeadDimV; ++j) {
3953     // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
3954     // reg[0] -> rows 0~7 的 {c0,c1}
3955     // reg[1] -> rows 8~15 的 {c2,c3}
3956     float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
3957     float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
3958     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
3959     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
3960
3961     // 0_new = exp(m_old - m_new) * 0_old + P@V (fused multiply-add)
3962     t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
3963     t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
3964     t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
3965     t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
3966
3967     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
3968     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);

```

```

3969     }
3970
3971     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
3972     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
3973     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
3974     lane_block_row_sum_old[i][0] = __fmaf_rn(
3975         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
3976     lane_block_row_sum_old[i][1] = __fmaf_rn(
3977         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
3978
3979     // Update m
3980     lane_block_row_max_old[i][0] = block_row_max_new_0;
3981     lane_block_row_max_old[i][1] = block_row_max_new_1;
3982 }
3983
3984 // Wait for next K tile to be ready in smem before next iteration
3985 if constexpr (kStage > 1) {
3986     if ((tile_K_seqlen + 1) < Tc) {
3987         CP_ASYNC_WAIT_GROUP(0);
3988     }
3989     __syncthreads();
3990 }
3991 } // end outer loop over K seqlen
3992 __syncthreads();
3993
3994 // =====
3995 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
3996 // =====
3997 #pragma unroll
3998 for (int i = 0; i < kValTileSeqLenP; ++i) {
3999     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
4000     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
4001     #pragma unroll
4002     for (int j = 0; j < kValTileHeadDimV; ++j) {
4003         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4004         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4005         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
4006         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
4007         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
4008         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
4009         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4010         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4011     }
4012 }
4013
4014 // =====
4015 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
4016 // =====
4017 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
4018 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
4019 #pragma unroll
4020 for (int i = 0; i < kValTileSeqLenP; ++i) {
4021     #pragma unroll
4022     for (int j = 0; j < kValTileHeadDimV; ++j) {
4023         uint32_t R_Z[2][4];
4024         R_Z[0][0] = R_D[i][j][0];
4025         R_Z[1][0] = R_D[i][j][1];
4026         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
4027         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
4028         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
4029         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
4030         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
4031         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);

```

```

4032 // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
4033 if (lane_id % 4 == 0) {
4034     int store_warp_regs_0_Br =
4035         warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
4036     int store_lane_gmem_0_Br =
4037         Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
4038     int store_warp_regs_0_d =
4039         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4040     int store_lane_gmem_0_d = store_warp_regs_0_d;
4041     int store_gmem_0_addr_0 = 0_gmem_offset +
4042         (store_lane_gmem_0_Br + 0) * kHeadDim +
4043         store_lane_gmem_0_d;
4044     int store_gmem_0_addr_1 = 0_gmem_offset +
4045         (store_lane_gmem_0_Br + 8) * kHeadDim +
4046         store_lane_gmem_0_d;
4047     // LDST128BITS = reinterpret_cast<float4*>
4048     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
4049         *reinterpret_cast<float4*>(&R_Z[0][0]);
4050     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
4051         *reinterpret_cast<float4*>(&R_Z[1][0]);
4052 }
4053 }
4054 }
4055 }
4056 }
4057
4058 // =====
4059 // 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
4060 // =====
4061
4062 static inline void check(cudaError_t err, const char *msg) {
4063     if (err != cudaSuccess) {
4064         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
4065         exit(EXIT_FAILURE);
4066     }
4067 }
4068
4069
4070 static void test_block_reduce(int N) {
4071
4072     srand(42);
4073     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4074     for (int i = 0; i < N; i++)
4075         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4076
4077     // CPU reference: sum of all elements
4078     double ref = 0.0;
4079     for (int i = 0; i < N; i++) ref += (double)h_a[i];
4080
4081     float *d_a, *d_y;
4082     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
4083     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
4084
4085     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
4086     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
4087
4088     dim3 block(128);
4089     dim3 grid((N + 127) / 128);
4090     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
4091     check(cudaGetLastError(), "blockreduce launch");
4092     check(cudaDeviceSynchronize(), "blockreduce sync");
4093
4094     float result;

```

```

4095 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
4096
4097 float err = fabsf(result - (float)ref);
4098 printf("| %-35s | %.6e | %-4s |\n", "BlockReduce", err,
4099     err < 1e-2f ? "PASS" : "FAIL");
4100
4101 free(h_a);
4102 cudaFree(d_a); cudaFree(d_y);
4103 }
4104
4105
4106 static void test_dot(int N) {
4107
4108     srand(42);
4109     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4110     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4111     for (int i = 0; i < N; i++) {
4112         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4113         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4114     }
4115
4116     // CPU reference
4117     double ref = 0.0;
4118     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4119
4120     float *d_a, *d_b, *d_y;
4121     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4122     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4123     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4124
4125     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4126     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4127     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4128
4129     dim3 block(128);
4130     dim3 grid((N + 127) / 128);
4131     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4132     check(cudaGetLastError(), "dot launch");
4133     check(cudaDeviceSynchronize(), "dot sync");
4134
4135     float result;
4136     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4137
4138     float err = fabsf(result - (float)ref);
4139     printf("| %-35s | %.6e | %-4s |\n", "Dot", err,
4140         err < 1e-2f ? "PASS" : "FAIL");
4141
4142     // ---- Dot Vec4 ----
4143     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4144     dim3 block_v4(32);
4145     dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
4146     check(cudaGetLastError(), "dot_vec4 launch");
4147     check(cudaDeviceSynchronize(), "dot_vec4 sync");
4148
4149     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4150     float err_v4 = fabsf(result - (float)ref);
4151     printf("| %-35s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
4152
4153     free(h_a); free(h_b);
4154     cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4155 }
4156
4157

```

```

4158 static void test_relu(int N) {
4159     srand(42);
4160     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4161     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4162     for (int i = 0; i < N; i++)
4163         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4164
4165     float *d_x, *d_y;
4166     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
4167     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
4168     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4169
4170     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4171     dim3 block256(256);
4172     dim3 grid256((N + 255) / 256);
4173     relu<<<grid256, block256>>>(d_x, d_y, N);
4174     check(cudaGetLastError(), "relu launch");
4175     check(cudaDeviceSynchronize(), "relu sync");
4176     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4177     float max_err = 0.0f;
4178     for (int i = 0; i < N; i++) {
4179         float expected = fmaxf(0.0f, h_x[i]);
4180         float err = fabsf(h_y[i] - expected);
4181         if (err > max_err) max_err = err;
4182     }
4183     printf("| %-35s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4184
4185     dim3 block64(64);
4186     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4187     check(cudaGetLastError(), "relu_vec4 launch");
4188     check(cudaDeviceSynchronize(), "relu_vec4 sync");
4189     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4190     max_err = 0.0f;
4191     for (int i = 0; i < N; i++) {
4192         float expected = fmaxf(0.0f, h_x[i]);
4193         float err = fabsf(h_y[i] - expected);
4194         if (err > max_err) max_err = err;
4195     }
4196     printf("| %-35s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4197
4198     free(h_x); free(h_y);
4199     cudaFree(d_x); cudaFree(d_y);
4200 }
4201
4202
4203 static void test_elementwise(int N) {
4204     srand(42);
4205     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4206     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4207     for (int i = 0; i < N; i++) {
4208         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4209         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4210     }
4211
4212     float *d_a, *d_b, *d_c;
4213     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4214     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4215     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4216     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4217     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4218
4219     dim3 block256(256);
4220     dim3 grid256((N + 255) / 256);

```

```

4221 elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4222 check(cudaGetLastError(), "eadd launch");
4223 check(cudaDeviceSynchronize(), "eadd sync");
4224 float *h_c = (float *)malloc((size_t)N * sizeof(float));
4225 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4226 float max_err = 0.0f;
4227 for (int i = 0; i < N; i++) {
4228     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4229     if (err > max_err) max_err = err;
4230 }
4231 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4232
4233 dim3 block64(64);
4234 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
4235 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4236 check(cudaGetLastError(), "eadd_vec4 launch");
4237 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
4238 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4239 max_err = 0.0f;
4240 for (int i = 0; i < N; i++) {
4241     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4242     if (err > max_err) max_err = err;
4243 }
4244 printf("| %-35s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4245
4246 free(h_a); free(h_b); free(h_c);
4247 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4248 }
4249
4250
4251 static void test_histogram(int N) {
4252     srand(42);
4253     int BINS = 16;
4254     int *h_hist = (int *)calloc(BINS, sizeof(int));
4255     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4256     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4257     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4258     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4259
4260     int *d_idx, *d_hist;
4261     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4262     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4263     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
4264     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4265
4266     dim3 block256(256);
4267     dim3 grid256((N + 255) / 256);
4268     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
4269     check(cudaGetLastError(), "histogram launch");
4270     check(cudaDeviceSynchronize(), "histogram sync");
4271     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
4272
4273     float max_err = 0.0f;
4274     for (int i = 0; i < BINS; i++) {
4275         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4276         if (err > max_err) max_err = err;
4277     }
4278     printf("| %-35s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4279
4280     free(h_hist); free(h_hist_ref); free(h_idx);
4281     cudaFree(d_idx); cudaFree(d_hist);
4282 }
4283

```



```

4284
4285 static void test_merge_attn_states(int num_tokens, int num_heads,
4286                                   int head_size) {
4287
4288     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4289     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4290
4291     float *h_prefix_out = (float *)malloc(out_size);
4292     float *h_suffix_out = (float *)malloc(out_size);
4293     float *h_prefix_lse = (float *)malloc(lse_size);
4294     float *h_suffix_lse = (float *)malloc(lse_size);
4295     float *h_output_ref = (float *)malloc(out_size);
4296
4297     srand(42);
4298     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4299         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4300         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4301     }
4302     for (int i = 0; i < num_heads * num_tokens; i++) {
4303         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4304         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4305     }
4306
4307     // CPU reference: 与 kernel 完全相同的浮点计算
4308     for (int t = 0; t < num_tokens; t++) {
4309         for (int h = 0; h < num_heads; h++) {
4310             float p_lse = h_prefix_lse[h * num_tokens + t];
4311             float s_lse = h_suffix_lse[h * num_tokens + t];
4312             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4313             s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4314
4315             float max_lse = fmaxf(p_lse, s_lse);
4316             p_lse -= max_lse;
4317             s_lse -= max_lse;
4318             float p_se = expf(p_lse);
4319             float s_se = expf(s_lse);
4320             float p_scale = p_se / (p_se + s_se);
4321             float s_scale = s_se / (p_se + s_se);
4322
4323             int head_off = t * num_heads * head_size + h * head_size;
4324             for (int d = 0; d < head_size; d++) {
4325                 h_output_ref[head_off + d] =
4326                     h_prefix_out[head_off + d] * p_scale +
4327                     h_suffix_out[head_off + d] * s_scale;
4328             }
4329         }
4330     }
4331
4332     float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4333     check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
4334     check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4335     check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4336     check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4337     check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4338
4339     check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4340                     cudaMemcpyHostToDevice),
4341           "merge_attn H2D p_out");
4342     check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4343                     cudaMemcpyHostToDevice),
4344           "merge_attn H2D s_out");
4345     check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4346                     cudaMemcpyHostToDevice),

```

```

4347     "merge_attn H2D p_lse");
4348 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4349                 cudaMemcpyHostToDevice),
4350        "merge_attn H2D s_lse");
4351
4352 int threads_per_head = head_size / 4;
4353 int total_threads = num_tokens * num_heads * threads_per_head;
4354 dim3 block(128);
4355 dim3 grid((total_threads + 127) / 128);
4356 merge_attn_states<<<grid, block>>>(<
4357     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4358     num_tokens, num_heads, head_size);
4359 check(cudaGetLastError(), "merge_attn launch");
4360 check(cudaDeviceSynchronize(), "merge_attn sync");
4361
4362 float *h_output = (float *)malloc(out_size);
4363 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4364        "merge_attn D2H");
4365
4366 float max_err = 0.0f;
4367 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4368     float err = fabsf(h_output[i] - h_output_ref[i]);
4369     if (err > max_err) max_err = err;
4370 }
4371 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
4372        max_err < 1e-4f ? "PASS" : "FAIL");
4373
4374 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
4375 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
4376 h_suffix_lse[0] = 0.0f;
4377 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4378                 cudaMemcpyHostToDevice),
4379        "merge_attn H2D inf lse");
4380 merge_attn_states<<<grid, block>>>(<
4381     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4382     num_tokens, num_heads, head_size);
4383 check(cudaGetLastError(), "merge_attn inf launch");
4384 check(cudaDeviceSynchronize(), "merge_attn inf sync");
4385 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4386        "merge_attn inf D2H");
4387
4388 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
4389 float inf_err = 0.0f;
4390 for (int d = 0; d < head_size; d++) {
4391     float err = fabsf(h_output[d] - h_suffix_out[d]);
4392     if (err > inf_err) inf_err = err;
4393 }
4394 printf("| %-35s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
4395        inf_err < 1e-4f ? "PASS" : "FAIL");
4396
4397 free(h_prefix_out);
4398 free(h_suffix_out);
4399 free(h_prefix_lse);
4400 free(h_suffix_lse);
4401 free(h_output_ref);
4402 free(h_output);
4403 cudaFree(d_prefix_out);
4404 cudaFree(d_suffix_out);
4405 cudaFree(d_prefix_lse);
4406 cudaFree(d_suffix_lse);
4407 cudaFree(d_output);
4408 }
4409

```

```

4410 static void test_softmax(int N) {
4411     // Use N = blockDim.x so one block processes all elements as one token.
4412     constexpr int kNumThreads = 256;
4413     if (N != kNumThreads) {
4414         printf("    OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
4415         return;
4416     }
4417
4418
4419     srand(42);
4420     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4421     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4422     for (int i = 0; i < N; i++)
4423         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
4424
4425     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4426     float max_val = -FLT_MAX;
4427     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4428     double sum_exp = 0.0;
4429     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4430     for (int i = 0; i < N; i++)
4431         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4432
4433     float *d_x, *d_y;
4434     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4435     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4436
4437     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4438
4439     dim3 block(256);
4440     dim3 grid(1); // one token covering all N elements
4441     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4442     check(cudaGetLastError(), "softmax launch");
4443     check(cudaDeviceSynchronize(), "softmax sync");
4444
4445     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4446     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4447
4448     float max_err = 0.0f;
4449     for (int i = 0; i < N; i++) {
4450         float err = fabsf(h_y[i] - h_y_ref[i]);
4451         if (err > max_err) max_err = err;
4452     }
4453     printf("| %-35s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4454
4455     // ---- Safe Softmax ----
4456     safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4457     check(cudaGetLastError(), "safe_softmax launch");
4458     check(cudaDeviceSynchronize(), "safe_softmax sync");
4459     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4460     max_err = 0.0f;
4461     for (int i = 0; i < N; i++) {
4462         float err = fabsf(h_y[i] - h_y_ref[i]);
4463         if (err > max_err) max_err = err;
4464     }
4465     printf("| %-35s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4466
4467     // ---- Naive Softmax ----
4468     softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4469     check(cudaGetLastError(), "naive_softmax launch");
4470     check(cudaDeviceSynchronize(), "naive_softmax sync");
4471     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4472     max_err = 0.0f;

```

```

4473     for (int i = 0; i < N; i++) {
4474         float err = fabsf(h_y[i] - h_y_ref[i]);
4475         if (err > max_err) max_err = err;
4476     }
4477     printf("| %-35s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4478
4479     free(h_x); free(h_y); free(h_y_ref);
4480     cudaFree(d_x); cudaFree(d_y);
4481 }
4482
4483
4484 static void test_rms_norm(int N, int K) {
4485
4486     srand(42);
4487     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4488     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4489     for (int i = 0; i < N * K; i++)
4490         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4491     float g = 1.5f; // gain
4492
4493     // CPU reference: y = (x / rms(x)) * g
4494     float epsilon = 1e-5f;
4495     for (int n = 0; n < N; n++) {
4496         double sum_sq = 0.0;
4497         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4498         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4499         for (int k = 0; k < K; k++)
4500             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4501     }
4502
4503     float *d_x, *d_y;
4504     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4505     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4506     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4507
4508     dim3 block(128);
4509     dim3 grid(N);
4510     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4511     check(cudaGetLastError(), "rmsnorm launch");
4512     check(cudaDeviceSynchronize(), "rmsnorm sync");
4513
4514     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4515     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4516
4517     float max_err = 0.0f;
4518     for (int i = 0; i < N * K; i++) {
4519         float err = fabsf(h_y[i] - h_y_ref[i]);
4520         if (err > max_err) max_err = err;
4521     }
4522     printf("| %-35s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4523
4524     // ---- RMS Norm Vec4 ----
4525     dim3 block_rv4(32);
4526     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4527     check(cudaGetLastError(), "rmsnorm_vec4 launch");
4528     check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4529     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4530     max_err = 0.0f;
4531     for (int i = 0; i < N * K; i++) {
4532         float err = fabsf(h_y[i] - h_y_ref[i]);
4533         if (err > max_err) max_err = err;
4534     }
4535     printf("| %-35s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");

```

```

4536     free(h_x); free(h_y); free(h_y_ref);
4537     cudaFree(d_x); cudaFree(d_y);
4538 }
4539
4540
4541
4542 static void test_layer_norm(int N, int K) {
4543
4544     srand(42);
4545     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4546     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4547     for (int i = 0; i < N * K; i++)
4548         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4549     float g = 1.5f, b = 0.3f; // gain and bias
4550
4551     // CPU reference: y = ((x - mean) / std) * g + b
4552     float epsilon = 1e-5f;
4553     for (int n = 0; n < N; n++) {
4554         double sum = 0.0;
4555         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4556         float mean = (float)sum / (float)K;
4557         double sum_sq = 0.0;
4558         for (int k = 0; k < K; k++) {
4559             float diff = h_x[n * K + k] - mean;
4560             sum_sq += (double)diff * (double)diff;
4561         }
4562         float std = sqrtf((float)sum_sq / (float)K + epsilon);
4563         for (int k = 0; k < K; k++)
4564             h_y_ref[n * K + k] = (h_x[n * K + k] - mean) / std * g + b;
4565     }
4566
4567     float *d_x, *d_y;
4568     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4569     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4570     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4571
4572     dim3 block(128);
4573     dim3 grid(N);
4574     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4575     check(cudaGetLastError(), "layernorm launch");
4576     check(cudaDeviceSynchronize(), "layernorm sync");
4577
4578     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4579     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4580
4581     float max_err = 0.0f;
4582     for (int i = 0; i < N * K; i++) {
4583         float err = fabsf(h_y[i] - h_y_ref[i]);
4584         if (err > max_err) max_err = err;
4585     }
4586     printf("| %-35s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4587
4588     // ---- Layer Norm Vec4 ----
4589     dim3 block_lv4(32);
4590     layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4591     check(cudaGetLastError(), "layernorm_vec4 launch");
4592     check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4593     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4594     max_err = 0.0f;
4595     for (int i = 0; i < N * K; i++) {
4596         float err = fabsf(h_y[i] - h_y_ref[i]);
4597         if (err > max_err) max_err = err;
4598     }

```

```

4599 printf("| %-35s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4600
4601 free(h_x); free(h_y); free(h_y_ref);
4602 cudaFree(d_x); cudaFree(d_y);
4603 }
4604
4605
4606 static void test_rope(int seq_len, int N) {
4607
4608     int total_pairs = seq_len * N;
4609     int total_elems = total_pairs * 2;
4610     size_t size = (size_t)total_elems * sizeof(float);
4611
4612     float *h_x = (float *)malloc(size);
4613     float *h_y_ref = (float *)malloc(size);
4614
4615     srand(42);
4616     for (int i = 0; i < total_elems; i++)
4617         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4618
4619     // CPU reference: 2D rotation for each pair
4620     for (int idx = 0; idx < total_pairs; idx++) {
4621         int token_pos = idx / N;
4622         int token_idx = idx % N;
4623         float x1 = h_x[idx * 2];
4624         float x2 = h_x[idx * 2 + 1];
4625         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4626         float angle = (float)token_pos * theta;
4627         float cos_v = cosf(angle);
4628         float sin_v = sinf(angle);
4629         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4630         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4631     }
4632
4633     float *d_x, *d_y;
4634     check(cudaMalloc(&d_x, size), "rope alloc X");
4635     check(cudaMalloc(&d_y, size), "rope alloc Y");
4636     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4637
4638     dim3 block(256);
4639     dim3 grid((total_pairs + 255) / 256);
4640     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
4641     check(cudaGetLastError(), "rope launch");
4642     check(cudaDeviceSynchronize(), "rope sync");
4643
4644     float *h_y = (float *)malloc(size);
4645     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
4646
4647     float max_err = 0.0f;
4648     for (int i = 0; i < total_elems; i++) {
4649         float err = fabsf(h_y[i] - h_y_ref[i]);
4650         if (err > max_err) max_err = err;
4651     }
4652     printf("| %-35s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4653
4654     free(h_x);
4655     free(h_y);
4656     free(h_y_ref);
4657     cudaFree(d_x);
4658     cudaFree(d_y);
4659 }
4660
4661

```



```

4662 static void test_mat_transpose(int row, int col) {
4663
4664     size_t size_in = (size_t)row * col * sizeof(float);
4665     size_t size_out = (size_t)col * row * sizeof(float);
4666
4667     float *h_x = (float *)malloc(size_in);
4668     float *h_y_ref = (float *)malloc(size_out);
4669
4670     srand(42);
4671     for (int i = 0; i < row * col; i++)
4672         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4673
4674     // CPU reference: y[j][i] = x[i][j] (row-major)
4675     for (int i = 0; i < row; i++)
4676         for (int j = 0; j < col; j++)
4677             h_y_ref[j * row + i] = h_x[i * col + j];
4678
4679     float *d_x, *d_y;
4680     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
4681     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
4682     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
4683
4684     dim3 block(16, 16);
4685     dim3 grid((col + 15) / 16, (row + 15) / 16);
4686     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
4687     check(cudaGetLastError(), "mattrans launch");
4688     check(cudaDeviceSynchronize(), "mattrans sync");
4689
4690     float *h_y = (float *)malloc(size_out);
4691     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
4692
4693     float max_err = 0.0f;
4694     for (int i = 0; i < col * row; i++) {
4695         float err = fabsf(h_y[i] - h_y_ref[i]);
4696         if (err > max_err) max_err = err;
4697     }
4698     printf("| %-35s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4699
4700     free(h_x);
4701     free(h_y);
4702     free(h_y_ref);
4703     cudaFree(d_x);
4704     cudaFree(d_y);
4705 }
4706
4707
4708 static void test_mat_transpose_padded(int row, int col) {
4709
4710     size_t size_in = (size_t)row * col * sizeof(float);
4711     size_t size_out = (size_t)col * row * sizeof(float);
4712
4713     float *h_x = (float *)malloc(size_in);
4714     float *h_y_ref = (float *)malloc(size_out);
4715
4716     srand(42);
4717     for (int i = 0; i < row * col; i++)
4718         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4719
4720     // CPU reference: y[j][i] = x[i][j] (row-major)
4721     for (int i = 0; i < row; i++)
4722         for (int j = 0; j < col; j++)
4723             h_y_ref[j * row + i] = h_x[i * col + j];
4724

```

```

4725 float *d_x, *d_y;
4726 check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
4727 check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
4728 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
4729
4730 dim3 block(16, 16);
4731 dim3 grid((col + 15) / 16, (row + 63) / 64);
4732 mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
4733 check(cudaGetLastError(), "mattrans_padded launch");
4734 check(cudaDeviceSynchronize(), "mattrans_padded sync");
4735
4736 float *h_y = (float *)malloc(size_out);
4737 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
4738
4739 float max_err = 0.0f;
4740 for (int i = 0; i < col * row; i++) {
4741     float err = fabsf(h_y[i] - h_y_ref[i]);
4742     if (err > max_err) max_err = err;
4743 }
4744 printf("| %-35s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
4745
4746 free(h_x);
4747 free(h_y);
4748 free(h_y_ref);
4749 cudaFree(d_x);
4750 cudaFree(d_y);
4751 }
4752
4753
4754 static void test_sgemv(int M, int K) {
4755
4756     srand(42);
4757     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
4758     float *h_x = (float *)malloc((size_t)K * sizeof(float));
4759     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4760     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4761     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4762
4763     // CPU reference: y[m] = sum_k A[m,k] * x[k]
4764     for (int m = 0; m < M; m++) {
4765         double sum = 0.0;
4766         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
4767         h_y_ref[m] = (float)sum;
4768     }
4769
4770     float *d_a, *d_x, *d_y;
4771     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
4772     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
4773     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
4774
4775     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
4776     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
4777
4778     dim3 block(32, 4);
4779     dim3 grid((M + 3) / 4);
4780     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
4781     check(cudaGetLastError(), "sgemv launch");
4782     check(cudaDeviceSynchronize(), "sgemv sync");
4783
4784     float *h_y = (float *)malloc((size_t)M * sizeof(float));
4785     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
4786
4787     float max_err = 0.0f;

```

```

4788 for (int m = 0; m < M; m++) {
4789     float err = fabsf(h_y[m] - h_y_ref[m]);
4790     if (err > max_err) max_err = err;
4791 }
4792 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4793
4794 // ---- SGEMV K32 ----
4795 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
4796 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
4797 check(cudaGetLastError(), "sgemv_k32 launch");
4798 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
4799
4800 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
4801
4802 max_err = 0.0f;
4803 for (int m = 0; m < M; m++) {
4804     float err = fabsf(h_y[m] - h_y_ref[m]);
4805     if (err > max_err) max_err = err;
4806 }
4807 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4808
4809 // ---- SGEMV K16 ----
4810 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4811 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4812
4813 int K16 = 16;
4814 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
4815 h_x = (float *)malloc((size_t)K16 * sizeof(float));
4816 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
4817 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4818 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4819
4820 for (int m = 0; m < M; m++) {
4821     double sum = 0.0;
4822     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
4823     h_y_ref[m] = (float)sum;
4824 }
4825
4826 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
4827 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
4828 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
4829
4830 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
4831 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
4832
4833 dim3 grid_k16((M + 7) / 8);
4834 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
4835 check(cudaGetLastError(), "sgemv_k16 launch");
4836 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
4837
4838 h_y = (float *)malloc((size_t)M * sizeof(float));
4839 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
4840
4841 max_err = 0.0f;
4842 for (int m = 0; m < M; m++) {
4843     float err = fabsf(h_y[m] - h_y_ref[m]);
4844     if (err > max_err) max_err = err;
4845 }
4846 printf("| %-35s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4847
4848 free(h_a); free(h_x); free(h_y); free(h_y_ref);
4849 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
4850 }

```

```

4851
4852
4853 static void test_sgemm(int M, int N, int K) {
4854
4855     size_t size_a = (size_t)M * K * sizeof(float);
4856     size_t size_b = (size_t)K * N * sizeof(float);
4857     size_t size_c = (size_t)M * N * sizeof(float);
4858
4859     float *h_a = (float *)malloc(size_a);
4860     float *h_b = (float *)malloc(size_b);
4861     float *h_c_ref = (float *)malloc(size_c);
4862
4863     srand(42);
4864     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4865     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4866
4867     float *d_a, *d_b, *d_c;
4868     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
4869     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
4870     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
4871
4872     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
4873     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
4874
4875     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
4876     cublasHandle_t handle;
4877     cublasCreate(&handle);
4878     float alpha = 1.0f, beta = 0.0f;
4879     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4880                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
4881     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
4882
4883     // Kernel
4884     cudaEvent_t start, stop;
4885     cudaEventCreate(&start);
4886     cudaEventCreate(&stop);
4887
4888     dim3 block(32, 32);
4889     dim3 grid((N + 31) / 32, (M + 31) / 32);
4890     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
4891     check(cudaGetLastError(), "sgemm launch");
4892     check(cudaDeviceSynchronize(), "sgemm sync");
4893
4894     float *h_c = (float *)malloc(size_c);
4895     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
4896
4897     // Verify
4898     float max_err = 0.0f;
4899     for (int i = 0; i < M * N; i++) {
4900         float err = fabsf(h_c[i] - h_c_ref[i]);
4901         if (err > max_err) max_err = err;
4902     }
4903     printf("| %-35s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4904
4905     // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
4906
4907     dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
4908     check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
4909     sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
4910     check(cudaGetLastError(), "sgemm_vec4 launch");
4911     check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
4912
4913     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");

```

```

4914 max_err = 0.0f;
4915 for (int i = 0; i < M * N; i++) {
4916     float err = fabsf(h_c[i] - h_c_ref[i]);
4917     if (err > max_err) max_err = err;
4918 }
4919 printf("| %-35s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
4920
4921 free(h_a); free(h_b); free(h_c); free(h_c_ref);
4922 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4923 cublasDestroy(handle);
4924 cudaEventDestroy(start);
4925 cudaEventDestroy(stop);
4926 }
4927
4928
4929
4930 static void test_hgemm_mma(int M, int N, int K) {
4931
4932     size_t size_a = (size_t)M * K * sizeof(half);
4933     size_t size_b = (size_t)K * N * sizeof(half);
4934     size_t size_c = (size_t)M * N * sizeof(half);
4935
4936     half *h_a = (half *)malloc(size_a);
4937     half *h_b = (half *)malloc(size_b);
4938     half *h_c_ref = (half *)malloc(size_c);
4939
4940     srand(42);
4941     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4942     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
4943
4944     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
4945     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
4946     size_t size_b_t = (size_t)N * K * sizeof(half);
4947     half *h_b_t = (half *)malloc(size_b_t);
4948     for (int n = 0; n < N; n++)
4949         for (int k = 0; k < K; k++)
4950             h_b_t[n * K + k] = h_b[k * N + n];
4951
4952     half *d_a, *d_b, *d_b_t, *d_c;
4953     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
4954     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
4955     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
4956     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
4957
4958     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
4959     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
4960     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
4961
4962     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
4963     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
4964     cublasHandle_t handle;
4965     cublasCreate(&handle);
4966     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
4967     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
4968                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
4969                 &beta_h, d_c, CUDA_R_16F, N,
4970                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
4971     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
4972
4973     // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
4974     cudaEvent_t start, stop;
4975     cudaEventCreate(&start);
4976     cudaEventCreate(&stop);

```

```

4977
4978 constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
4979 size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
4980 dim3 block(256);
4981 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
4982 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
4983 check(cudaGetLastError(), "hgemm launch");
4984 check(cudaDeviceSynchronize(), "hgemm sync");
4985
4986 half *h_c = (half *)malloc(size_c);
4987 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
4988
4989 // Verify
4990 float max_err = 0.0f;
4991 for (int i = 0; i < M * N; i++) {
4992     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
4993     if (err > max_err) max_err = err;
4994 }
4995 printf("| %-35s | %.6e | %-4s |\n", "HGMEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
4996
4997 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
4998 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
4999 cublasDestroy(handle);
5000 cudaEventDestroy(start);
5001 cudaEventDestroy(stop);
5002 }
5003
5004
5005 static void test_hgemm_swizzle(int M, int N, int K) {
5006     // HGMEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
5007     // + Register Double Buffering (kValTileK=2, BK=32)
5008     // TN layout: C[M×N] = A[M×K] × BT[N×K]
5009     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
5010     // (kValTileK=2, kStages=3, BK=32)
5011     // smem: kStages × (BM×BK + BN×BK) halves
5012
5013     size_t size_a = (size_t)M * K * sizeof(half);
5014     size_t size_b = (size_t)K * N * sizeof(half);
5015     size_t size_c = (size_t)M * N * sizeof(half);
5016
5017     half *h_a = (half *)malloc(size_a);
5018     half *h_b = (half *)malloc(size_b);
5019     half *h_c_ref = (half *)malloc(size_c);
5020
5021     srand(42);
5022     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5023     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5024
5025     // Kernel expects BT [N×K] row-major layout (TN layout convention).
5026     size_t size_b_t = (size_t)N * K * sizeof(half);
5027     half *h_b_t = (half *)malloc(size_b_t);
5028     for (int n = 0; n < N; n++)
5029         for (int k = 0; k < K; k++)
5030             h_b_t[n * K + k] = h_b[k * N + n];
5031
5032     half *d_a, *d_b, *d_b_t, *d_c;
5033     check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
5034     check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
5035     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
5036     check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
5037
5038     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
5039     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");

```



```

5040 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
5041
5042 // cuBLAS FP16 reference
5043 cublasHandle_t handle;
5044 cublasCreate(&handle);
5045 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5046 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5047             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5048             &beta_h, d_c, CUDA_R_16F, N,
5049             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5050 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
5051
5052 // MMA swizzle kernel (default params: kStages=3, kValTileK=2, BK=32)
5053 constexpr int BM = 128, BN = 128, BK = 32, K_STAGE_S = 3;
5054 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
5055 dim3 block(256);
5056 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5057 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5058 check(cudaGetLastError(), "hgemm_swizzle launch");
5059 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
5060
5061 half *h_c = (half *)malloc(size_c);
5062 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
5063
5064 // Verify
5065 float max_err = 0.0f;
5066 for (int i = 0; i < M * N; i++) {
5067     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5068     if (err > max_err) max_err = err;
5069 }
5070 printf("| %-35s | %.6e | %-4s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "FAIL");
5071
5072 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5073 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5074 cublasDestroy(handle);
5075 }
5076
5077
5078 #if defined(NOTES_V2_ENABLE_CUTE)
5079 static void test_hgemm_cute(int M, int N, int K) {
5080     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3> + kStage=2
5081     // TN layout: C[MxN] = A[MxK] × B^T[NxK]
5082     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
5083     // Tile: BM=128, BN=256, BK=32, 128 threads/block
5084
5085     size_t size_a = (size_t)M * K * sizeof(half);
5086     size_t size_b = (size_t)K * N * sizeof(half);
5087     size_t size_c = (size_t)M * N * sizeof(half);
5088
5089     half *h_a = (half *)malloc(size_a);
5090     half *h_b = (half *)malloc(size_b);
5091     half *h_c_ref = (half *)malloc(size_c);
5092
5093     srand(42);
5094     for (int i = 0; i < M * K; i++)
5095         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5096     for (int i = 0; i < K * N; i++)
5097         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5098
5099     // CuTe kernel expects B^T [NxK] row-major (TN layout).
5100     size_t size_b_t = (size_t)N * K * sizeof(half);
5101     half *h_b_t = (half *)malloc(size_b_t);
5102     for (int n = 0; n < N; n++)

```

```

5103     for (int k = 0; k < K; k++)
5104         h_b_t[n * K + k] = h_b[k * N + n];
5105
5106     half *d_a, *d_b, *d_b_t, *d_c;
5107     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
5108     check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
5109     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
5110     check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
5111
5112     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
5113     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
5114     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
5115
5116     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5117     cublasHandle_t handle;
5118     cublasCreate(&handle);
5119     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5120     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5121                  CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5122                  CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5123     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
5124
5125     // CuTe kernel (Stages=2, TN layout)
5126     launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
5127     check(cudaGetLastError(), "hgemm_cute launch");
5128     check(cudaDeviceSynchronize(), "hgemm_cute sync");
5129
5130     half *h_c = (half *)malloc(size_c);
5131     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
5132
5133     // Verify
5134     float max_err = 0.0f;
5135     for (int i = 0; i < M * N; i++) {
5136         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5137         if (err > max_err) max_err = err;
5138     }
5139     printf("| %-35s | %.6e | %-4s |\n", "HGEMM CuTe", max_err, max_err < 1.0f ? "PASS" : "FAIL");
5140
5141     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5142     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5143     cublasDestroy(handle);
5144 }
5145 #endif /* NOTES_V2_ENABLE_CUTE */
5146
5147
5148 #if defined(NOTES_V2_ENABLE_WGMMMA)
5149 static void test_hgemm_wgmma(int M, int N, int K) {
5150     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
5151     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
5152     // Kernel: hgemm_wgmma_stages_tn with default template params
5153
5154     constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;
5155
5156     // M, K must be divisible by tile dims
5157     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5158         printf("| %-35s | %-12s | %-4s |\n",
5159              "HGEMM WGMMMA", "SKIP", "SKIP");
5160         return;
5161     }
5162
5163     size_t size_a = (size_t)M * K * sizeof(half);
5164     size_t size_b = (size_t)K * N * sizeof(half);
5165     size_t size_c = (size_t)M * N * sizeof(half);

```

```

5166
5167     half *h_a = (half *)malloc(size_a);
5168     half *h_b = (half *)malloc(size_b);
5169     half *h_c_ref = (half *)malloc(size_c);
5170
5171     srand(42);
5172     for (int i = 0; i < M * K; i++)
5173         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5174     for (int i = 0; i < K * N; i++)
5175         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5176
5177     // B^T [N×K] row-major for TN layout (same as hgemm_mma kernel)
5178     size_t size_b_t = (size_t)N * K * sizeof(half);
5179     half *h_b_t = (half *)malloc(size_b_t);
5180     for (int n = 0; n < N; n++)
5181         for (int k = 0; k < K; k++)
5182             h_b_t[n * K + k] = h_b[k * N + n];
5183
5184     half *d_a, *d_b, *d_b_t, *d_c;
5185     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5186     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5187     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5188     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5189
5190     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5191     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5192            "wgmma H2D B (cuBLAS)");
5193     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5194            "wgmma H2D B_t (kernel)");
5195
5196     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
5197     cublasHandle_t handle;
5198     cublasCreate(&handle);
5199     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5200     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5201                  CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5202                  CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5203     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5204            "wgmma D2H ref");
5205
5206     // Create TMA tensor maps for A and B^T
5207     // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5208     // B^T[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
5209     CUsensorMap *tma_a =
5210         allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5211     CUsensorMap *tma_b =
5212         allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5213
5214     // Launch WGMMMA kernel
5215     // kBlockSwizzle=false → 2D grid, no swizzle
5216     size_t smem_bytes =
5217         kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
5218     cudaFuncSetAttribute(
5219         hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
5220         false>,
5221         cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5222
5223     dim3 block(kNumThreads);
5224     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5225     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
5226         <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5227     check(cudaGetLastError(), "wgmma launch");
5228     check(cudaDeviceSynchronize(), "wgmma sync");

```

```

5229
5230 half *h_c = (half *)malloc(size_c);
5231 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5232
5233 // Verify
5234 float max_err = 0.0f;
5235 for (int i = 0; i < M * N; i++) {
5236     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5237     if (err > max_err)
5238         max_err = err;
5239 }
5240 printf("| %-35s | %.6e | %-4s |\n", "HGEMM WGMMMA", max_err,
5241        max_err < 1.0f ? "PASS" : "FAIL");
5242
5243 free(h_a);
5244 free(h_b);
5245 free(h_b_t);
5246 free(h_c);
5247 free(h_c_ref);
5248 cudaFree(d_a);
5249 cudaFree(d_b);
5250 cudaFree(d_b_t);
5251 cudaFree(d_c);
5252 cudaFree(tma_a);
5253 cudaFree(tma_b);
5254 cublasDestroy(handle);
5255 }
5256 #endif /* NOTES_V2_ENABLE_WGMMMA */
5257
5258
5259 static void test_flash_attn(int seqlen, int head_dim) {
5260     // FlashAttention-2 with split-Q, MMA m16n8k16
5261     int B = 1, H = 8;
5262
5263     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5264
5265     srand(42);
5266     half *h_q = (half *)malloc(sz);
5267     half *h_k = (half *)malloc(sz);
5268     half *h_v = (half *)malloc(sz);
5269     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5270         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5271         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5272         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5273     }
5274
5275     // CPU reference in FP32:  $O = \text{softmax}(Q @ K^T / \sqrt{d}) @ V$ 
5276     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
5277     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5278     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5279     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5280     int count = B * H * seqlen * head_dim;
5281     for (int i = 0; i < count; i++) {
5282         ref_q[i] = __half2float(h_q[i]);
5283         ref_k[i] = __half2float(h_k[i]);
5284         ref_v[i] = __half2float(h_v[i]);
5285     }
5286
5287     float scale = 1.0f / sqrtf((float)head_dim);
5288     for (int bi = 0; bi < B * H; bi++) {
5289         for (int qi = 0; qi < seqlen; qi++) {
5290             //  $S[qi, kj] = Q[qi, :] @ K[kj, :]^T * \text{scale}$ 
5291             float smax = -INFINITY;

```

```

5292 float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5293 for (int kj = 0; kj < seqlen; kj++) {
5294     float s = 0.0f;
5295     for (int d = 0; d < head_dim; d++)
5296         s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5297             ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5298     S[kj] = s * scale;
5299     if (S[kj] > smax) smax = S[kj];
5300 }
5301 // softmax
5302 double sum_exp = 0.0;
5303 for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5304 float inv_sum = 1.0f / (float)sum_exp;
5305 // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
5306 for (int d = 0; d < head_dim; d++) {
5307     double o_acc = 0.0;
5308     for (int kj = 0; kj < seqlen; kj++)
5309         o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5310             ref_v[bi * seqlen * head_dim + kj * head_dim + d];
5311     ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5312 }
5313 free(S);
5314 }
5315 }
5316
5317 half *d_q, *d_k, *d_v, *d_o;
5318 check(cudaMalloc(&d_q, sz), "fa alloc Q");
5319 check(cudaMalloc(&d_k, sz), "fa alloc K");
5320 check(cudaMalloc(&d_v, sz), "fa alloc V");
5321 check(cudaMalloc(&d_o, sz), "fa alloc O");
5322 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5323 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5324 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5325
5326 // Template params for kHeadDim=64, kStage=2
5327 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
5328 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
5329 constexpr int kMmaTileSeqLenQ = 4;
5330 constexpr int kMmaTileSeqLenK = 1;
5331 constexpr int kMmaTileSeqLenP = 4;
5332 constexpr int kMmaTileHeadDimV = 1;
5333 constexpr int kValTileSeqLenQ = 1;
5334 constexpr int kValTileSeqLenK = 8;
5335 constexpr int kValTileSeqLenP = 1;
5336 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5337
5338 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5339 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
5340 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
5341                     kStageV * Bc * (kHeadDim + kPadV) +
5342                     Bc * (kHeadDim + kPadV)) * sizeof(half);
5343
5344 dim3 block(128);
5345 dim3 grid((seqlen + Br - 1) / Br, B * H);
5346
5347 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
5348     kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
5349     kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
5350     kStageV, kPadV>
5351     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
5352 check(cudaGetLastError(), "fa launch");
5353 check(cudaDeviceSynchronize(), "fa sync");
5354

```

```

5355     half *h_o = (half *)malloc(sz);
5356     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
5357
5358     float max_err = 0.0f;
5359     for (int i = 0; i < count; i++) {
5360         float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
5361         if (err > max_err) max_err = err;
5362     }
5363     printf("| %-35s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
5364
5365     free(h_q); free(h_k); free(h_v); free(h_o);
5366     free(ref_q); free(ref_k); free(ref_v); free(ref_o);
5367     cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
5368 }
5369
5370
5371 int main(int argc, char *argv[]) {
5372     #if defined(NOTES_V2_ENABLE_WGMMMA)
5373         cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
5374     #endif
5375     int M = 1024, N = 1024, K = 1024;
5376     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
5377
5378     printf("=== notes-v2.cu verification harness ===\n");
5379     printf("| %-35s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
5380     printf("|-----|-----|-----|\n");
5381
5382     test_block_reduce(N);
5383     test_dot(N);
5384     test_relu(1024);
5385     test_elementwise(1024);
5386     test_histogram(1024);
5387     test_merge_attn_states(512, 16, 128);
5388     test_softmax(256);
5389     test_rms_norm(8, 128);
5390     test_layer_norm(8, 128);
5391     test_rope(8, 128);
5392     test_mat_transpose(256, 256);
5393     test_mat_transpose_padded(256, 256);
5394     test_sgemv(256, 128);
5395     test_sgemm(M, N, K);
5396     test_hgemmm_mma(M, N, K);
5397     test_hgemmm_swizzle(M, N, K);
5398     #if (defined(NOTES_V2_ENABLE_CUTE))
5399         test_hgemmm_cute(M, N, K);
5400     #endif
5401     #if (defined(NOTES_V2_ENABLE_WGMMMA))
5402         test_hgemmm_wgmma(M, N, K);
5403     #endif
5404     test_flash_attn(1024, 64);
5405
5406     printf("=== All tests done ===\n");
5407     return 0;
5408 }
5409
5410 // =====
5411 // Quick build & run reference
5412 // =====
5413 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
5414 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
5415 // -I ../../third-party/cutlass/include \
5416 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
5417 //

```



```

5418 // # sm_89 单独编译 + 运行:
5419 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
5420 //
5421 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
5422 // 项目内置 third-party/cutlass) :
5423 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
5424 // -I ../../third-party/cutlass/include \
5425 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
5426 //
5427 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
5428 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5429 // -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
5430 //
5431 // # sm_90a + CuTe + WGMMA:
5432 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5433 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
5434 // -I ../../third-party/cutlass/include \
5435 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin

```